Robot Rubik's Cube Solver

By

Peter Myler

This report is submitted in partial fulfilment of the requirements of the Honours Degree in Electrical and Electronic Engineering (TU821) of

Technological University Dublin

May 25th, 2026

Supervisor: Frank Duignan

Abstract

This report describes the design, construction, and evaluation of an autonomous robotic system capable of solving a scrambled 3×3×3 Rubik's cube. The robot consists of a custom 3D-printed mechanical frame holding six stepper motors, each connected directly to one face of the cube via a custom 3D-printed motor coupler. Two USB webcams capture images of all visible facelets, which are processed using HSV-based colour detection to construct the cube's current state. Kociemba's two-phase algorithm computes a near-optimal solution sequence, which is transmitted from the host computer to an Arduino Nano microcontroller over a serial link. The microcontroller drives the six stepper motors through stepper drivers to execute the solution. A custom graphical user interface allows the user to monitor the camera feeds, adjust system parameters, calibrate the colour detection, and execute random scrambles and solves.

The completed system solves a scrambled cube successfully on approximately 99% of attempts, with the remaining failures attributable to occasional misclassification by the colour detection stage under varying lighting conditions. Successful full solves, which include the colour-detection, solution computation and physical execution stages, complete in an average time of 2.13 seconds, with a fastest recorded solve of just 1.5 seconds – significantly faster than the current official human world record of 2.76 seconds [23]. The project demonstrates the integration of mechanical design, electronics, embedded systems, computer vision, and software into a single autonomous system, and provides a practical platform for the demonstration of applied robotics.

Declaration

I confirm that the content of this document is my own original work, except where otherwise acknowledged. This includes all text, figures, tables, schematics, and images, etc.

Furthermore, I declare that this document, and the work described herein, has not been submitted for a degree or other award at any other university or institution.

I confirm that I am familiar with the university's policy on appropriate use of artificial intelligence https://tudublin.libguides.com/GenAI/guidelinesresources.

Generative AI was not used during this project.

Student Signature: Peter Myler

Date: 25/05/2026

Table of Contents

1 Introduction

The Rubik's Cube is one of the most recognisable mechanical puzzles in the world. Despite its simple appearance, the number of possible scrambled states exceeds 43 quintillion, higher than the total number of grains of sand on Earth. Solving it autonomously requires combining mechanical design, electronics, embedded systems, computer vision, and software into a single interconnected system.

The aim of this project was to design and build a fully autonomous Rubik's Cube solving robot. The system allows a user to scramble a standard 3×3×3 Rubik's Cube by hand, place it into the robot, and start the solving process through a graphical user interface. After that, no further input or manual help is needed. The robot identifies the current state of the cube, computes a solution, and executes the moves automatically.

The robot consists of a custom 3D-printed frame holding six stepper motors, each rotating one face of the cube. Custom motor couplers connect each motor directly to a face of the cube. Two USB webcams positioned at opposite corners of the robot capture images of all six faces, which are processed using HSV-based colour detection to determine the state of the cube. Kociemba's two-phase algorithm then computes a near-optimal solution, which is sent over a serial link to an Arduino Nano microcontroller. The Arduino drives the six stepper motors through stepper drivers to physically execute the solution on the cube. The host-side software, written in Python, runs on a PC or laptop and provides a graphical user interface for monitoring, calibrating, testing, and controlling the robot.

The primary objectives of the project were:

- To design and build a robust mechanical frame capable of holding six stepper motors and securing the Rubik's Cube during operation.
- To implement accurate and reliable stepper motor control using a microcontroller.
- To develop a computer vision algorithm capable of detecting the colour of each facelet and determining the full cube state.
- To integrate an efficient solving algorithm and translate its output into executable motor commands.
- To develop a graphical user interface for user control and real-time system monitoring.
- To combine all subsystems into a fully autonomous and reliable robotic system.

The remainder of this report is structured as follows:

- Chapter 2 covers the ethical considerations relevant to the project.

- Chapter 3 provides background research on the key technical areas behind the system, including cube-solving algorithms, motor and driver selection, computer vision approaches, and a review of existing designs.
- Chapter 4 describes the overall system architecture and the key design decisions. Chapter 5 covers the implementation of each subsystem in detail.
- Chapter 6 documents the testing and results.
- Chapter 7 presents the conclusions and ideas for future work.

2 Ethics

This chapter covers the ethical considerations relevant to the design, construction, and use of the Rubik's Cube solving robot. The project does not involve human subjects, clinical applications, or the processing of personal data, but a few ethical considerations are still relevant.

2.1 Health and Safety

The robot involves mechanical motion, electrical power, and the use of workshop tools during construction, all of which introduce potential safety risks.

During construction, soldering, wiring, and general assembly were carried out following standard laboratory safety practices, including working in a well-ventilated area and using appropriate personal protective equipment. The 3D printer used to manufacture the mechanical parts was operated following its standard operating procedure, with attention paid to ventilation and unattended operation.

During operation, the six stepper motors rotate the faces of the cube at significant speed and torque, which introduces a risk of finger entrapment if the mechanism is started while hands are inside the frame. To mitigate this, the system only starts a solve through the user interface, after the user has placed the cube and moved their hands clear. The electrical system runs at 14 V with a 4.8 A current limit. Idle current draw is around 0.7 A, rising to roughly 2–3 A during operation. All components run within their rated voltage and current limits, and appropriate insulation and strain relief are used throughout the wiring.

2.2 Professional and Technical Responsibility

The project was developed following accepted engineering practice, including proper documentation, structured testing, and iterative validation of each subsystem. As set out in Engineers Ireland's Code of Ethics, engineers are expected to take responsibility for the quality and safety of their work and to act with integrity in all professional matters [24].

The most significant external dependency in the project is the *RubiksCube-TwophaseSolver* library by Herbert Kociemba [4]. This library is published under the GNU General Public Licence v3.0 (GPL-3.0), which permits free use, modification, and distribution as long as any derivative work is distributed under the same licence. The library has been used strictly in accordance with these terms. It has not been modified and is properly referenced throughout this documentation. No other external libraries with restrictive licensing terms were used.

All design decisions, implementation choices, and engineering judgements were the independent work of the student, with guidance from the project supervisor. The system was designed to behave predictably and safely.

2.3 Data Protection and Privacy

The project uses two cameras to capture images of the Rubik's Cube during the colour detection stage. These cameras are pointed only at the cube within the robot's frame and cannot capture images of people in normal operation. Captured images are processed in memory and are not saved to disk or stored beyond the duration of a single solve cycle. Since the system does not collect, store, or transmit any personal data, the General Data Protection Regulation (GDPR) does not apply to this project. This was a deliberate design choice, to keep the system simple and free of any data protection obligations.

2.4 Societal and Educational Impact

Although this project is primarily a technical and educational exercise, it shows the integration of mechanical design, electronics, embedded systems, computer vision, and software into a single autonomous robotic system, a combination of disciplines that is increasingly relevant in modern engineering.

The system could serve as a useful demonstration platform for illustrating concepts in robotics, control systems, and computer vision to undergraduate students. In this respect, the project contributes to the 4th United Nations Sustainable Development Goal [25]: to ensure inclusive and equitable access to quality education and promote lifelong learning opportunities for all by providing a tangible, accessible demonstration of applied engineering.

2.5 Use of Artificial Intelligence

No artificial intelligence tools were used at any stage of this project, including during software development, the design of the mechanical and electronic systems, or the preparation of this documentation. All technical work, implementation, design decisions, and written content are the independent work of the student.

3 Background and Research

The development of an autonomous Rubik's Cube solving robot requires research across several technical areas, including cube-solving algorithms, mechanical design, motor selection, embedded systems, computer vision, and serial communication. Existing documentation, open-source libraries, project build logs, and video demonstrations of similar systems were reviewed to inform the design decisions made for this project [3]-[23]. This chapter summarises the key findings.

3.1 Rubik's Cube Terminology and Notation

Before discussing the algorithms and engineering challenges involved in solving a Rubik's Cube autonomously, it's useful to establish the terminology used throughout this report.

The standard Rubik's Cube has 26 individual physical pieces called cubelets, arranged in a 3×3×3 grid around an internal mechanism that allows each face to rotate independently (the cubelet in the exact centre of the grid is not counted). These cubelets fall into three categories: 8 corner cubelets (each with three visible coloured faces), 12 edge cubelets (each with two visible coloured faces), and 6 centre cubelets (each with a single coloured visible face). The individual coloured surfaces on each cubelet are called facelets rather than "stickers". The arrangement of all facelets across the cube at any given moment is referred to as the current cube state.

Slice moves rotate one or more of the inner layers rather than the outer faces. However, any slice move is geometrically equivalent to rotating the two outer faces parallel to it in the opposite direction. As a result, slice moves don't add any extra capability beyond the six standard face rotations. This is why a six-motor robot, with one motor per face, is mechanically sufficient to perform every possible cube manipulation. The colour of each centre therefore defines the colour of its face, as well as the orientation of the cube as a whole. This property has important implications for cube-solving algorithms and, as discussed later, for camera-based cube state detection.

Face rotations are described using Singmaster notation [21] (Appendix B), which assigns a single letter to each of the six outer faces: U (Up), D (Down), L (Left), R (Right), F (Front), and B (Back). A single letter (e.g. R) means a *90° clockwise* rotation when looking straight onto the face. An apostrophe (e.g. R') means a *90° anticlockwise* rotation, and a numeric suffix of 2 (e.g. R2) means a *half-turn* or *180° rotation*.

It is also important to note the distinction between "moves" and "turns". Moves are defined as any face rotation regardless of the angle, so R, R', and R2 all each count as 1 move. Turns are defined in terms of how many half-turns were made, so R and R' each count as 1 turn and

R2 is counted as 2 turns. This means that even though God's number is 20 moves, it is actually closer to 28 turns on average.

3.2 Kociemba's Two-Phase Algorithm

A key part of any cube-solving system is the algorithm used to compute the solution. Several approaches exist, ranging from layer-by-layer methods to computationally efficient near-optimal solvers. The layer-by-layer method known as CFOP is used by the vast majority of the speed-solving community. While it's a great method for a human, it can require over 60 moves to solve a properly scrambled cube. It has been proved that any scramble can be solved in 20 moves or less [22], which is known as God's Number. Since CFOP usually requires well over that move count, it is impractical for a fast robotic system.

Kociemba's two-phase algorithm is a well-documented method that solves this by breaking the solution process into two constrained search phases [3]. In the first phase, the algorithm manipulates the cube into an intermediate state called G1, in which all corners and edges are oriented correctly and all middle-layer edges are in the middle layer. From this state, the second phase solves the rest of the cube using a restricted set of moves: 180° half-turns for the F, B, L, and R faces, combined with any rotation of the U and D faces. By restricting the second phase like this, the search space is dramatically reduced, allowing efficient computation of a near-optimal solution.

The algorithm uses large precomputed lookup tables to guide the search, requiring around 80 MB of storage and taking some time to generate on first use [4]. After that initial cost, solutions of around 22 moves can be found almost instantly. An iterative deepening search is used to progressively find shorter solutions within a given time budget, although this feature went mostly unused in the project itself as the time limit given to the solver was quite short.

An open-source Python implementation of this algorithm is available as an installable package [4] and maintained by Herbert Kociemba himself! Using a proven and well-tested library allowed the project to focus on system integration and reliability rather than algorithm development from scratch.

3.3 Existing Designs

Several existing Rubik's Cube solving robots were examined through project build logs and video demonstrations [5]-[11]. These sources provided practical insight into mechanical and electrical design choices, common implementation challenges, and the relative merits of different approaches.

Kostoski [5] documents the iterative development of a six-motor stepper-based solver. The build log gives useful insight into the practical challenges of motor-to-cube coupling and the

importance of frame rigidity, and also highlights the limitations of lower-torque motors – an important consideration during motor selection for this project.

Musa [6], [7] presents a similar six-motor design using NEMA 17 steppers driven by TMC2208 silent stepper drivers. All his parts were 3D-printed in PLA at 20% infill. The project shows a mechanical solution for motor-to-cube coupling using custom-printed adapter pieces. A later video by the same author [11] shows an upgraded design using brushless DC motors with magnetic encoders, achieving significantly faster solve times.

Flatland [8] demonstrates one of the fastest stepper-based robotic solvers publicly documented, showing the upper bounds of what's achievable with a well-tuned design.

3.4 Alternative Designs

Alternative mechanical architectures were also considered. The CUBOTino project [9] and a dual-axis design by HiDe Tech [10] both use fewer than six motors, instead rotating the entire cube to bring different faces into a position where they can be manipulated. While these mechanisms are interesting demonstrations of what can be done with minimal hardware, they add significant complexity to both the mechanical design and the software control logic.

These designs are also much slower as repositioning the entire cube between moves adds significant dead time that isn't spent directly executing moves on the cube. They are also more prone to alignment errors.

It's also worth mentioning that it is theoretically possible to solve a Rubik's Cube by being able to turn only five of the faces. However, this approach requires up to 13 extra moves of the other faces just to rotate the stationary face once, drastically increasing solve time.

Given the project's goals of reliability, speed, and mechanical simplicity, these reduced-motor approaches were rejected. The six-motor architecture was selected as it maps each motor directly to one face, simplifies the control logic, and allows any move sequence produced by the solver to be executed directly.

3.5 Motor Selection

Motor selection was an important design decision affecting cost, control complexity, solve speed, and reliability. Two main options were considered: stepper motors and brushless DC (BLDC) motors with encoders.

BLDC motors can reach very high rotational speeds and are used in the highest-performance cube-solving machines [11]. However, they require more complex control electronics, closed-loop feedback via encoders, and careful tuning to be reliable. The extra cost and complexity

make BLDC motors less suitable for a first-time integrated robotics project where reliability and simplicity are prioritised.

Stepper motors were selected due to their inherent positional accuracy, straightforward interface with microcontrollers via step and direction signals through drivers, and predictable behaviour. NEMA 17 motors in particular were identified through the reviewed projects [5]-[6] as the most reliable choice for this application. The specific motor selected was the 17HS4401 [12], offering 40 N·cm of holding torque and a 1.8° step angle. Slower and lower torque alternatives such as the 28BYJ-48 stepper motor were found to produce inconsistent rotations and were unsuitable [5].

3.6 Stepper Motor Drivers

Stepper motors cannot be driven directly from a microcontroller. They require a dedicated driver circuit to convert step-and-direction logic signals into the higher-current coil patterns needed to rotate the motor. The two most common driver ICs in hobbyist robotics are the A4988 and the TMC2208.

The A4988 is a long-established and inexpensive driver capable of supplying up to roughly 2 amps per coil [14]. It's well suited to lower-cost projects but uses a simple chopper drive that produces very loud coil whine. It can also lose synchronisation at high step rates, causing missed steps and inaccurate rotations.

The TMC2208 is a more modern silent stepper driver developed by Trinamic Motion Control, widely used in 3D printers for its quiet operation [13]. It uses StealthChop, which drives the motor coils smoothly with voltage-mode control rather than the abrupt current chopping used by the A4988. This produces much less noise and noticeably smoother motion.

3.7 Colour Detection

Reliable identification of the cube's current state is needed for computing a correct solution. The reviewed projects use two broad approaches: manual input, where the user enters the colour of each tile through an interface, or automated camera-based detection [5]-[11].

Manual input methods avoid the technical challenges of computer vision entirely but rely on the user to correctly identify and enter each of the 54 facelet colours. This is error-prone and adds significant time to the overall solve. Camera-based detection lets the system operate much faster and is, most importantly, fully autonomous.

The reviewed designs that implement camera-based detection use traditional image processing techniques like colour-space analysis, rather than machine learning. This is appropriate for the controlled environment in which the robot operates, where lighting and

camera positions are mostly consistent. HSV (hue, saturation, value) colour space methods are well suited to this kind of environment, are computationally lightweight, and don't require training data. For these reasons, a camera-based HSV detection approach was selected for this project.

3.8 Serial Communication

In a system where the solving algorithm runs on a host computer and motor control runs on a dedicated microcontroller, a reliable communication interface between the two is required. There are two general options available: serial link over a physical cable and Bluetooth. A serial link provides better reliability, simplicity, and no pairing overhead. Therefore, a standard direct serial connection over a USB cable was used in this project.

3.9 Cube State Detection with Partial Visibility

A practical limitation of any six-motor cube-solving design is that the motor couplers occupy the position of the centre facelet on each face. Six facelets (one per face) are physically obscured and cannot be seen by any external camera. Depending on the geometry of the couplers and camera positions, the couplers may also block some corner facelets adjacent to each face. This raises a question that needs to be addressed during design: if some facelets can't be seen directly, how can the full cube state be determined?

Fortunately, this is easily solvable in both cases! Since slice moves are not used – the centre piece of each face is fixed relative to the rest of the cube, meaning they cannot be moved by any sequence of face rotations. The colour of each centre facelet is therefore determined entirely by the cube's known orientation in the robot (green as Front and white as Up).

As for the hidden corner facelets: a Rubik's Cube has eight corner pieces, each with a unique combination and relative orientation of three facelet colours. If two of the three facelets on a corner are visible, the third can be determined by identifying which of the eight possible corner pieces the visible pair belongs to. The orientation of the corner, which is set by the relative spatial arrangement of the two visible facelets, tells you which colour the hidden facelet is.

Together these techniques allow a six-motor design to use just two cameras to detect the entire cube state.

4 System Design and Architecture

This chapter describes the overall architecture of the system and the key design decisions made for each subsystem before implementation. The goal is to give a complete picture of what was designed and why, before the implementation detail is covered in Chapter 5.

4.1 System Architecture Overview

The system has three main hardware elements: a host computer, an Arduino Nano microcontroller, and the mechanical frame itself, which houses the motors and USB webcams. A high-level block diagram is shown below:

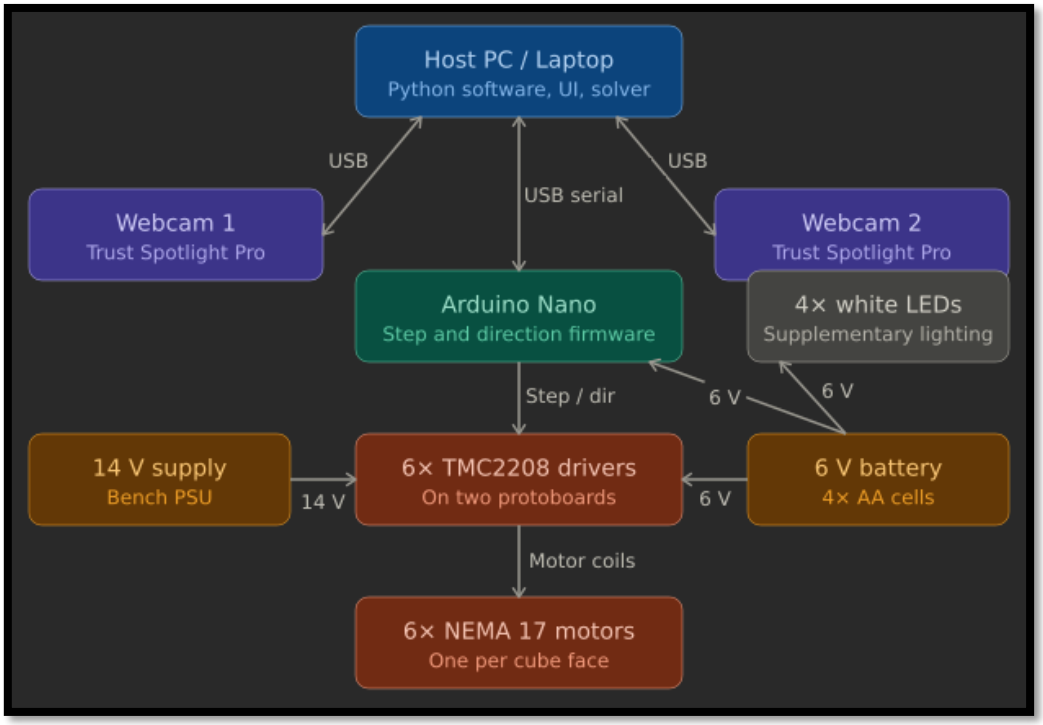


Figure 4-1. System block diagram

The host computer handles all high-level functionality: it runs the user interface, captures images from the two webcams, performs colour detection, runs the Kociemba two-phase solver, and sends the resulting move sequence to the Arduino Nano. The microcontroller receives the move sequence, interprets each move, and generates step and direction signals for the six stepper drivers, which in turn drive the six stepper motors. Once the full sequence has been executed, the microcontroller sends an acknowledgement back to the host.

This division of responsibilities was a deliberate design choice. The Kociemba algorithm has substantial computational and memory requirements making it unsuitable for a low-power

microcontroller. Generating accurate step-and-direction signals at hundreds of microsecond intervals requires precise real-time timing, which is far easier on a dedicated microcontroller than on a general-purpose operating system. Splitting the work this way lets each device do what it's best suited to.

4.2 Mechanical Design

The mechanical design uses the six-motor architecture established in Chapter 3, with one stepper motor mounted on each face of a cubic frame, oriented to drive the corresponding face of the cube. The frame is fully 3D-printed in PLA and held together with M3 screws and nuts throughout, so any part can be replaced or modified easily.

Motor-to-cube coupling. Two approaches were considered. The first, used in some existing designs, is to insert 3D-printed adapters into the cube's centre caps [6]. This was rejected because it significantly modifies the cube itself and makes it uncomfortable to scramble by hand. The other approach is to remove the centre facelet of each face (which is a simple cover) and design a coupler that fits into the recessed cavity beneath. This keeps the cube's physical shape and feel while providing a strong mechanical connection to the motor shaft. The coupler is octagonal in profile to match the cavity, with a cylinder containing a D-shaped hole that fits the flat on the stepper motor shaft.

Frame rigidity and cube insertion. A fully rigid frame would prevent the cube from being inserted between the motors, since the cube and the couplers need to be in contact. Some mechanical compliance is therefore needed. The four side mounts (Left, Right, Front, Back) can flex slightly outwards to allow the cube to be placed inside, but must be held rigidly in place during operation to keep the motor shafts aligned. To achieve this, the top mount interlocks with the four side mounts, locking them in place once fitted. Additional 3D-printed brackets clamp the top down to the sides to prevent any movement during operation.

Print orientation for structural integrity. An early version of the base with all four side mounts as a single part was printed, which left the individual layer lines with a very small surface area. Two sides snapped cleanly along the layer lines under some slight bending.

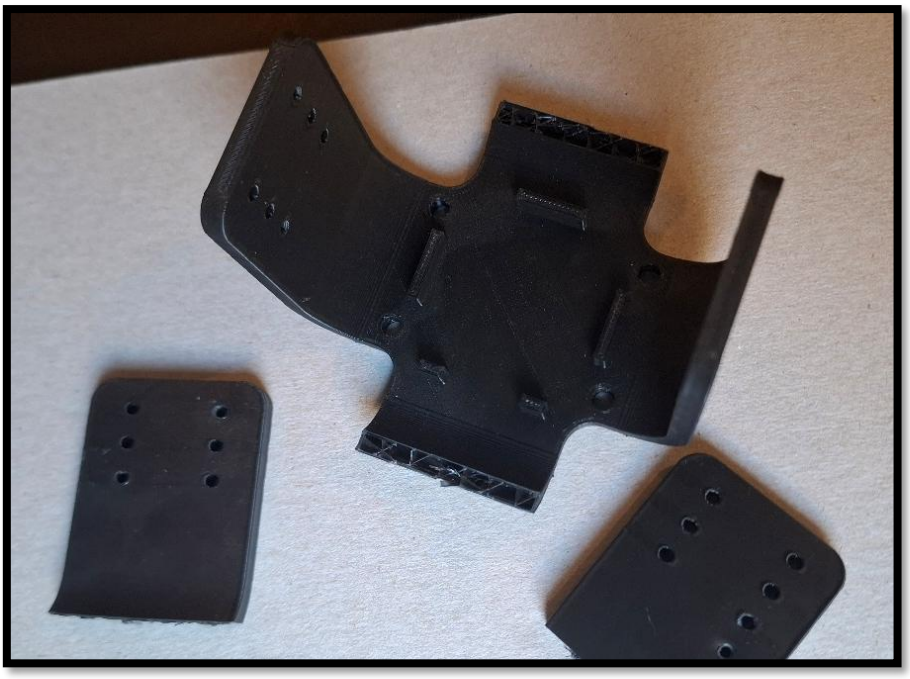


Figure 4-2. Failed 3D-printed base

The revised design splits the base into two pieces, allowing each to be printed on its side so the layer lines have much higher surface area and run perpendicular to the stress direction. This significantly increased the amount this part could flex and resolved the snapping problem. Heat-set inserts were used to join the parts together and bolt the bottom motor.

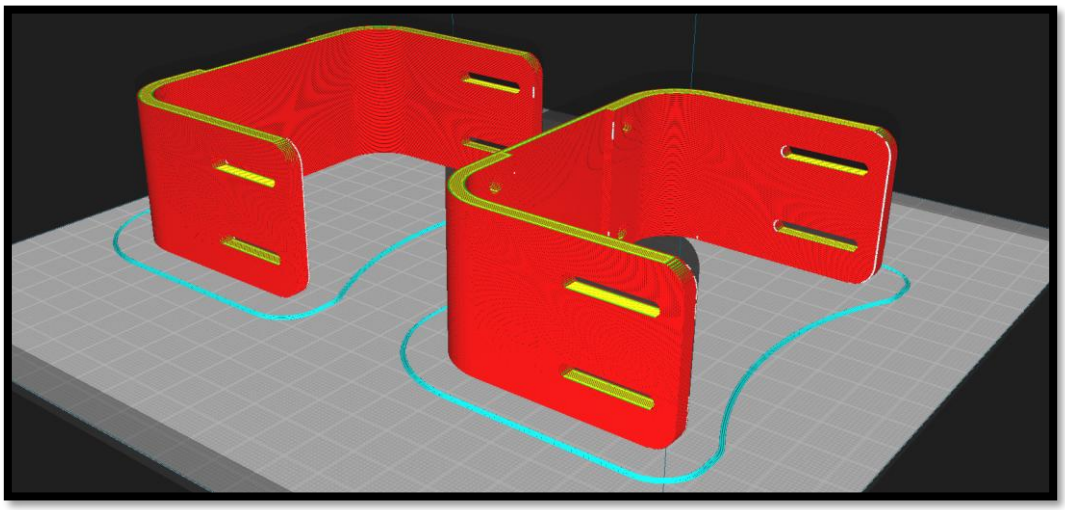


Figure 4-3. Revised base pieces

4.3 Electronics Design

The electronics subsystem covers the microcontroller, the six stepper motor drivers, the power supply, and the supplementary lighting needed for reliable colour detection.

Microcontroller. An Arduino Nano [15] was selected as the motor controller. It provides enough digital I/O for the twelve step-and-direction signals needed by the six drivers, has well-supported development tools, and can be programmed and powered over a single USB connection from the host. More capable microcontrollers were considered, but the project didn't need anything beyond the Nano's capabilities.

Stepper drivers. Initial testing used A4988 drivers, which are widely available and inexpensive. In practice these produced loud audible coil whine, ran the motors more slowly than expected, and behaved unreliably at higher step rates, occasionally missing steps. The drivers were replaced with TMC2208 silent stepper drivers, which resolved all three issues: operation became almost silent, achievable step rates were higher, and rotations were consistently accurate. The TMC2208 is pin-compatible with the A4988, so the swap was straightforward. The lowest microstepping setting on the TMC2208 was selected, since the system only ever performs 90° or 180° rotations.

Power supply. The motors are powered by a variable bench power supply set to 14 V with the current limit at 4.8 A. Although idle current draw is around 0.7 A and normal operation is around 2-3 A, the initial inrush current of all six motors at startup is much higher, and a lower limit prevented the motors from initialising properly. The Arduino and supplementary lighting are powered separately from a 6 V battery pack (four AA batteries).

Supplementary lighting. Although the chosen webcams have integrated white LEDs around the lens, in practice these don't provide enough or sufficiently uniform illumination to reliably detect colours across all visible facelets. Four extra white LEDs were added: two for the top camera's field of view, and two for the bottom. These ensure that all visible faces are more evenly lit.

4.4 Vision System Design

The vision subsystem captures images of the cube and determines its current state. Two USB Trust Spotlight Pro webcams are positioned at opposite corners of the robot so that all six faces are visible across the two cameras combined. These were selected for their low cost, adjustable mounting, and integrated lighting.

Colour space selection. An initial implementation used the mean RGB value of each facelet's image region. This was unreliable because RGB couples colour information with brightness – the same red facelet can produce significantly different RGB values under different lighting, leading to misclassifications even with the supplementary lighting. The system was moved to HSV (hue, saturation, value), which separates hue from brightness and saturation, allowing classification to be based primarily on hue and made far more robust to lighting variation.

Statistical measure. Within each facelet's region, a statistical measure of the pixel colour is needed. The mean was tried first but was found to be sensitive to outlier pixels caused by edge effects, glare, dust, or imperfections in the printed facelet boundaries. The median was found to be much more robust and was therefore adopted.

Region definition. Each facelet's region is defined by a quadrilateral, a four-vertex polygon, with each vertex being able to be positioned independently. Quadrilaterals were chosen rather than rectangles because the cameras view the cube from oblique angles, and the apparent shape of each facelet in the image is a perspective-projected quadrilateral rather than a square. Defining the region this way allows accurate sampling without needing any image rectification.

Hidden facelet recovery. As discussed in Section 3.9, some facelets are blocked by the motor couplers. The six centre facelets are filled in deterministically based on the cube's known orientation. The obscured corner facelets are recovered by identifying which of the eight unique corner cubelets the visible pair belongs to, and using the spatial arrangement of those two facelets to determine the orientation of the corner and therefore the colour of the hidden third facelet.

4.5 Software Architecture

The host computer software has four main parts: the user interface, the camera capture and colour detection pipeline, the solver interface, and the serial communication module.

The user interface is implemented using CustomTkinter [17] and runs on the main thread. It displays the live camera feeds, presents controls for testing and calibration, and starts solves. The camera capture pipeline, implemented using the OpenCV library [16], runs continuously in the background, providing the most recent frame from each camera on demand. When a solve is started, the most recent frames are passed to the colour detection module, which extracts the cube state and passes it to the solver. The solver returns a move sequence, which is translated from Kociemba notation to standard cube notation and then sent to the Arduino over the serial link using pyserial [18].

One design consideration is that the serial read used to wait for the Arduino's acknowledgement is a blocking operation. If this were done on the main thread, it would freeze the entire UI, including the camera feeds, for the duration of the solve. To prevent this, the serial communication is handled asynchronously on a separate thread, letting the main thread keep updating the UI and camera feeds while a solve is in progress.

4.6 Microcontroller Firmware Design

The Arduino Nano firmware receives a move sequence as a single transmission over the serial link, parses it into individual moves, and executes each move by generating step and direction signals for the relevant stepper driver. Once the full sequence has been executed, the firmware sends an acknowledgement back to the host with the executed sequence and the total execution time.

Two parameters control execution speed: the inter-step delay, which determines how quickly the motor runs, and the inter-move delay, which adds a settling time between consecutive moves. Both are exposed to the user through the host UI so they can be adjusted during testing. The system was found to operate reliably with an inter-step delay of 320 µs and zero inter-move delay; shorter step delays caused the motors to lose synchronisation.

5 Implementation

This chapter describes the implementation of each subsystem of the robot, building on the design decisions established in Chapter 4. It's organised by subsystem, starting with the mechanical build, followed by the electronics, the Arduino firmware, the vision pipeline, and finally the host-side software and user interface. A full bill of materials for the system is provided in Appendix C.

5.1 Mechanical Build

The mechanical assembly is made up of around 20 individual 3D-printed parts, all printed in black PLA at 0.2 mm layer height on a Creality Ender-3 Neo. Most parts use 20% infill with two perimeter walls, with higher infill and more walls used selectively for parts subject to greater mechanical load or that need to flex during cube insertion. Most structural parts are 5 mm thick. The completed assembly is around 190 × 190 × 190 mm in size, not counting the camera mounts which stick out about another 100mm on opposite sides of the frame.

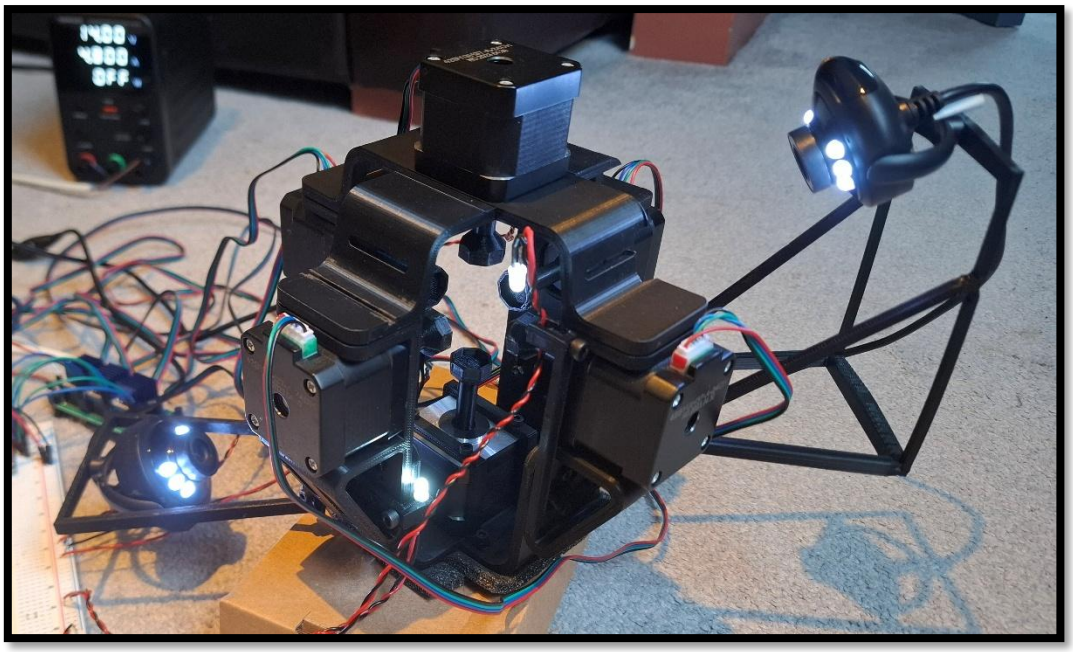


Figure 5-1. Completed assembly

The frame is held together using M3 screws and nuts, allowing any part to be replaced or modified with ease. This was valuable during development, when several parts had to be redesigned and reprinted to fix structural or spacing issues.

Motor couplers. Each motor is connected to its cube face via a custom coupler that fits into the cavity left after removing the cube's centre facelet. The coupler is 33 mm long and has two main features: an octagonal head profiled to match the recessed cavity in the centre face of the cube, and an internal cylinder with a D-shaped hole that fits perfectly onto the stepper motor shaft. The design went through several iterations before achieving a good fit. Early versions either snapped, didn't fit into the cube centre, or sat too loosely on the shaft. The final design provides a sturdy connection between motor and cube face.

Figure 5-2. Motor coupler design iterations

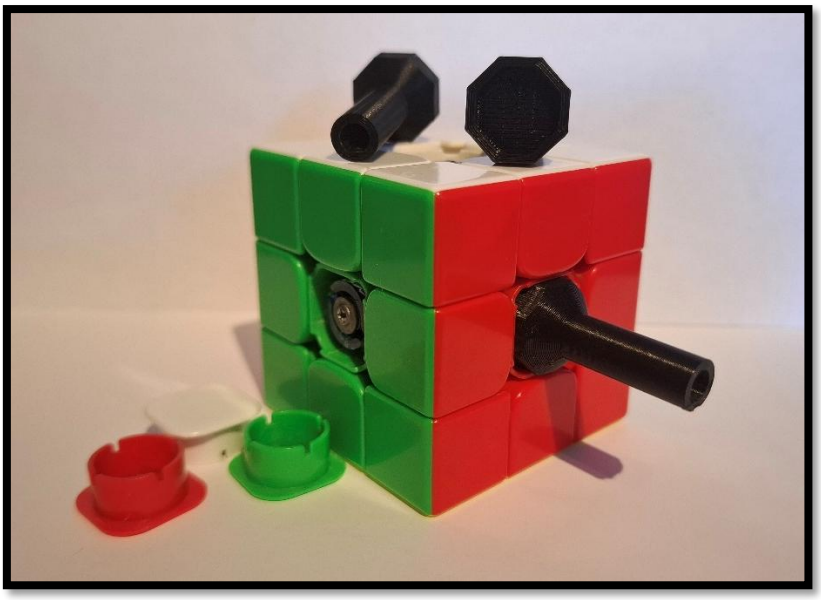


Figure 5-3. Motor couplers and cube

Mechanical frame. The frame was designed in Fusion 360. It consists of about 20 individual parts that join together with M3 nuts and bolts. The base consists of two separate U-shaped parts. The bottom stepper motor sits in the centre and is clamped down by two parts. Side mounts attach to each of the four sides of the base, which house the side stepper motors. The top part also consists of two parts joined together which sit on-top of the side mounts and are secured using some bracket parts to allow for easy disassembly. The top motor is bolted to the top parts.

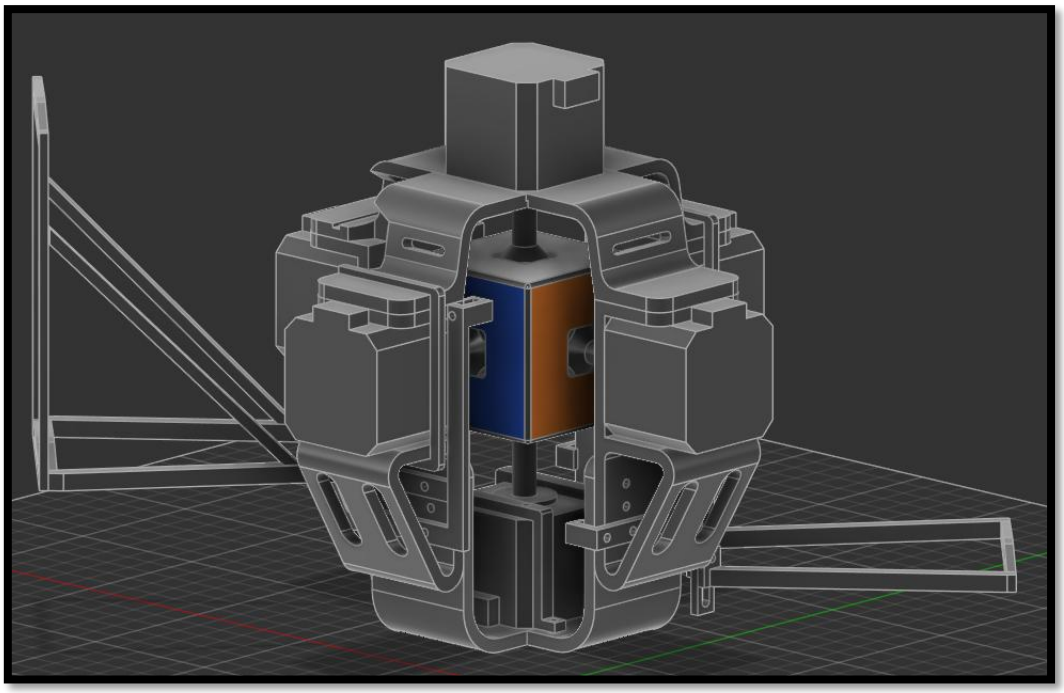


Figure 5-4. Fusion 360 model

Cube insertion. To insert the cube into the robot the top mount can simply be lifted up after removing the clamps that hold it down. The four side mounts can then be flexed slightly outwards to make room for the cube. The cube must be oriented with the white face up and the green face forwards. Physical markers on the frame show the correct orientation. Once the cube is seated against the bottom and all four side couplers, the top mount is put back and clamped down.

5.2 Electronics

The electronics are organised across three connected boards: a single breadboard with the Arduino Nano and the supplementary LEDs, and two protoboards each carrying three TMC2208 stepper drivers. Motor power comes from a NANKADF WPS3010H variable bench power supply set to 14 V with the current limit at 4.8 A. The Arduino and the supplementary LEDs are powered separately from a 6 V battery pack (four AA batteries).

Power distribution. The 14 V from the bench supply goes directly into the motor supply (VM) pin of each of the six TMC2208 drivers, providing the higher-current motor coil supply. The 6 V battery pack provides power to the Arduino (through its Vin pin), the TMC2208 logic supplies, and the four supplementary LEDs. Although the system draws around 0.7 A at idle and 2–3 A during normal operation, the simultaneous inrush current of all six motors at startup is significantly higher; the 4.8 A limit accommodates this transient without folding back the supply.

Driver protoboards. Each protoboard holds three TMC2208 drivers, with each driver sitting in soldered pin headers rather than being soldered directly. This lets a driver be swapped out easily if it fails (a useful feature given that a few TMC2208s were destroyed during development). The microstepping select pins are tied to ground on each driver, putting the TMC2208 in its lowest microstepping setting. The control pins are wired to another set of pin headers so that they can be connected with jumper wires to the Arduino. The motor coil pins are connected to a set of male pin headers which allow the wire which came with each stepper motor to be connected.

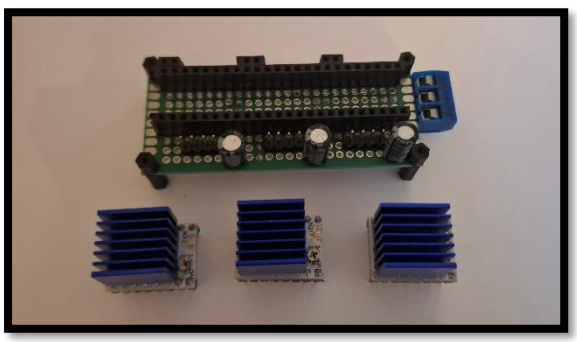


Figure 5-5. Driver protoboard without drivers

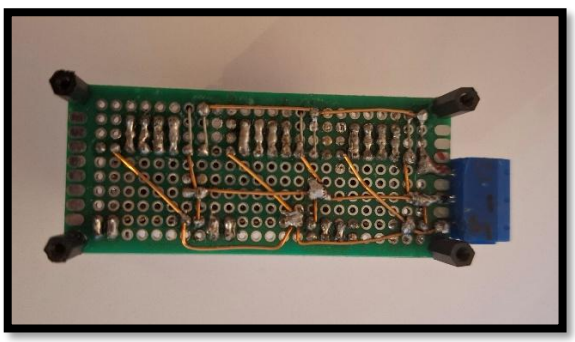


Figure 5-6. Driver protoboard underside

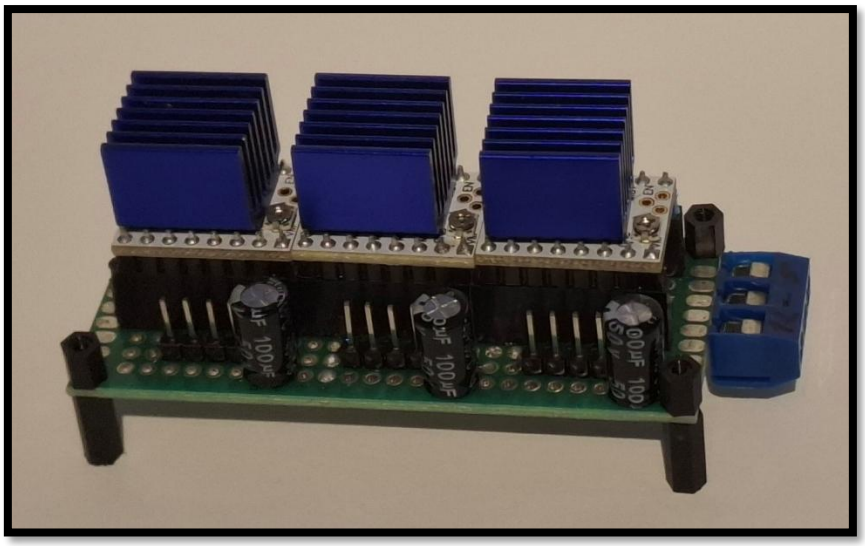


Figure 5-7. Driver protoboard

Wiring to the Arduino. Each driver needs two control signals from the Arduino: a step pulse and a direction signal. Twelve digital outputs are therefore needed in total. Pins 2 to 13 are used, with each driver receiving a step pin and a direction pin on consecutive numbers (driver 0 uses pins 2 and 3, driver 1 uses pins 4 and 5, and so on). Female pin connectors on each protoboard accept loose wires that run back to the breadboard, keeping the assembly modular. The mapping between motor index and cube face is handled in firmware via a lookup table.

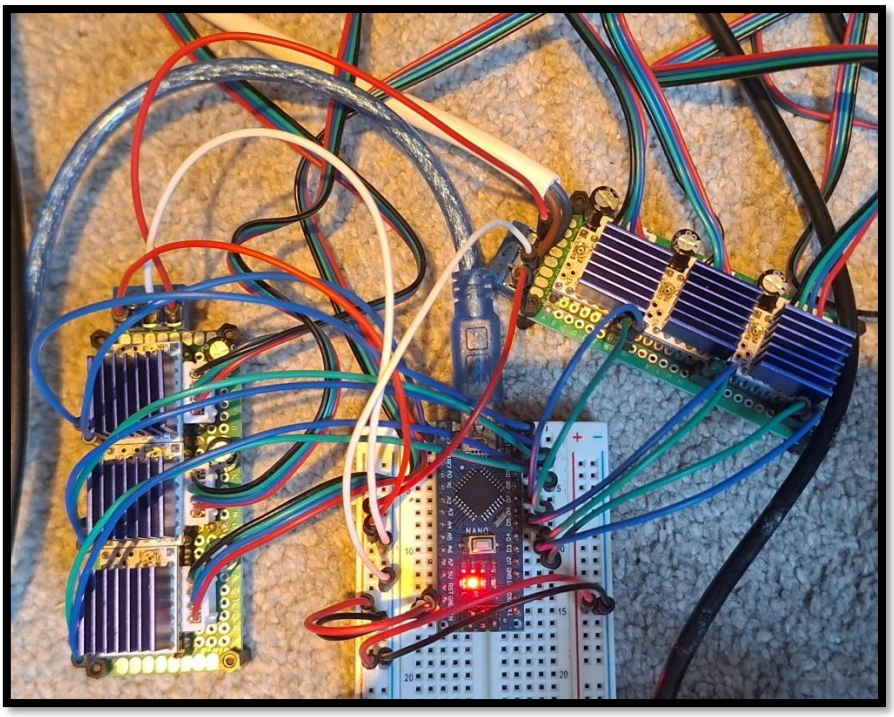


Figure 5-8. Drivers and Arduino

Supplementary lighting. Four extra white LEDs were added to provide enough illumination for colour detection – two for the top camera's field of view and two for the bottom. The chosen webcams have integrated LEDs around the lens, but these alone don't provide enough or sufficiently uniform light to reliably distinguish the six colours, especially at the oblique camera angles needed to see multiple faces. The supplementary LEDs are wired directly to the 6V battery rail on the breadboard.

5.3 Arduino Firmware

The Arduino Nano firmware is a single sketch of around 170 lines that handles all real-time motor control (Appendix A). The firmware is deliberately minimal, with no external libraries beyond the standard Arduino package.

Pin mapping and motor naming. Six pairs of step and direction pins are declared at the top of the sketch, alongside an array mapping motor index 0 to 5 to the corresponding face character (D, F, B, R, L, U). All face-name lookups go through a small search function which returns the motor index for a given face character, or -1 if the character isn't recognised.

Step generation. Each 90° face rotation takes 100 step pulses (a 2× microstepping configuration on the 1.8°-per-step). A 180° rotation doubles this to 200 steps. Each step pulse is generated by setting the step pin HIGH for stepDelay microseconds, then LOW for another stepDelay microseconds, so each full step takes 2 × stepDelay. With the default stepDelay of 320 µs, the step rate is approximately 1560 steps per second. A constant step rate is used throughout each move. No acceleration ramping is implemented.

Move parsing. The main loop blocks on Serial.available() and reads incoming commands one line at a time. If the first character of the command matches a known face name, the command is treated as a move sequence; otherwise, it's parsed as a parameter update. For move sequences, the firmware iterates through the command string, accumulating characters into a per-move buffer and dispatching each move on encountering a space. Each move is one or two characters: a face name optionally followed by an apostrophe (anticlockwise) or the digit 2 (180° rotation). The default direction is clockwise.

Parameter updates. Sending two integers separated by a space (for example, "320 100") updates the delay in-between steps (speed) and the delay between moves (delay) parameters in real time. This lets the host easily adjust the speed and delay values of the motors without having to restart or reprogram the Arduino.

Acknowledgement. Once an entire move sequence is done, the firmware computes the total execution time using millis() and sends a single acknowledgement line back to the host containing the elapsed time, the number of moves, and the equivalent turn count.

5.4 Vision Pipeline – Camera.py

The vision subsystem runs entirely in Python on the host, using OpenCV [16] for camera capture and image processing. It's implemented in *Camera.py* (Appendix A), with each of the two cameras represented by an instance of the *Camera.Camera* class (Appendix H-1).

Camera configuration. Cameras are connected to by using their respective Device Id stored in the json file (Appendix G). Each Camera instance is configured with a fixed resolution of 640×480, a hardware exposure setting, and a buffer size of zero to ensure that frames returned are always the most recent (Appendix H-1).

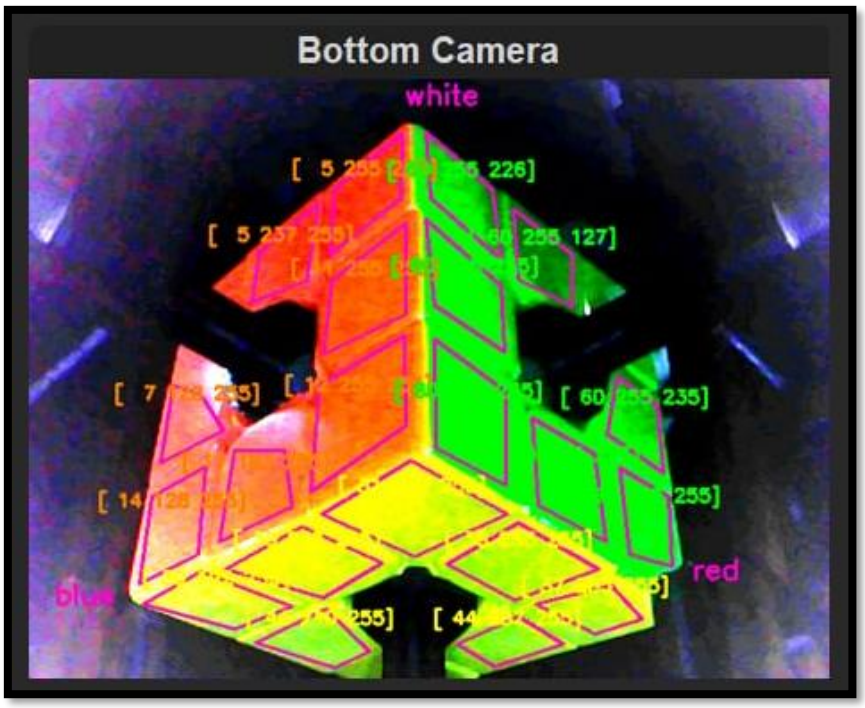


Figure 5-9. Bottom camera colour analysis

Frame preprocessing. Each frame is converted from BGR to RGB, flipped upside-down if necessary, and passed through cv2.normalize to expand its contrast (Appendix H-2) as the chosen webcams produce relatively dim images even with the supplementary lighting.

Quadrilateral sampling. Each visible facelet is associated with a quadrilateral defined by four vertex pixel coordinates stored in *cube_data.json* (Appendix G). The original implementation used simple small rectangular boxes positioned on each facelet, but this left a lot of their surface area unused. Switching to four-vertex quadrilaterals lets almost the entire visible surface of each facelet be sampled, significantly reducing the impact of grain and reflections on the classification result.

Hue-based classification. The classification function takes a single HSV tuple and returns one of the six colours (red, orange, yellow, green, blue, or white). The base classification uses a

prewritten array of seven hue boundaries that partition the hue space into ranges for each colour (Appendix H-3). Unfortunately, hue alone isn't always enough to classify colours under the lighting conditions. White detection is done first, before any hue-based classification, because white facelets have low saturation and aren't reliably distinguished by hue alone. Red and orange are then disambiguated within the low-hue range using the value channel. A similar split is applied between yellow and green. These special cases were arrived at with some trial-and-error during testing. The implementation can be viewed in Appendix H-4.

Hidden facelet recovery. As discussed in Section 3.9, the motor couplers physically obscure some corner facelets on each camera. These are calculated by matching against a prewritten array (Appendix H-5) containing the colours of each of the eight corner cubelets, listed in top, right, front order. By knowing two colours of a corner piece and their relative order the third can be determined by iterating through the array and finding a match.

5.5 Cube State and Solver Integration – Cube.py

The construction of the full 54-character cubestring and the interaction with the Kociemba solver are handled in *Cube.py* (Appendix A).

The cubestring is built by mapping the visible facelet colours, the recovered hidden corner facelets, and the deterministic centre facelets into the index positions expected by the Kociemba library, using prewritten index arrays (Appendix D, Appendix E, Appendix I-2).

The cubestring is passed to the solver [4] (Appendix I-4) and is given a 0.1 second time limit to find the lowest-move solve it can. The solver can return an "Error" string, meaning the cubestring is invalid. This signals an incorrect reading in the colour detection stage. A correction algorithm (Appendix I-5) is ran which can correct for a single incorrect colour reading. If this correction fails the entire solve attempt is deemed as a failure.

5.6 Host Software and User Interface – GUI.py

The host software is split across three Python modules plus a JSON configuration file (Appendix A). *GUI.py* (Appendix J) is the main script, containing the App class which implements the user interface using CustomTkinter [17], as well as handling most of the general functionality. The file *cube_data.json* (Appendix G) holds the camera device IDs, motor speed and inter-move delay values, and the full set of quadrilateral coordinates for both cameras. Using an external configuration file separates these parameters from the source code, allowing them to be changed and saved without modifying the codebase.

UI structure. The user interface is built around a single App class that subclasses customtkinter.CTk. The window is split into two main regions:

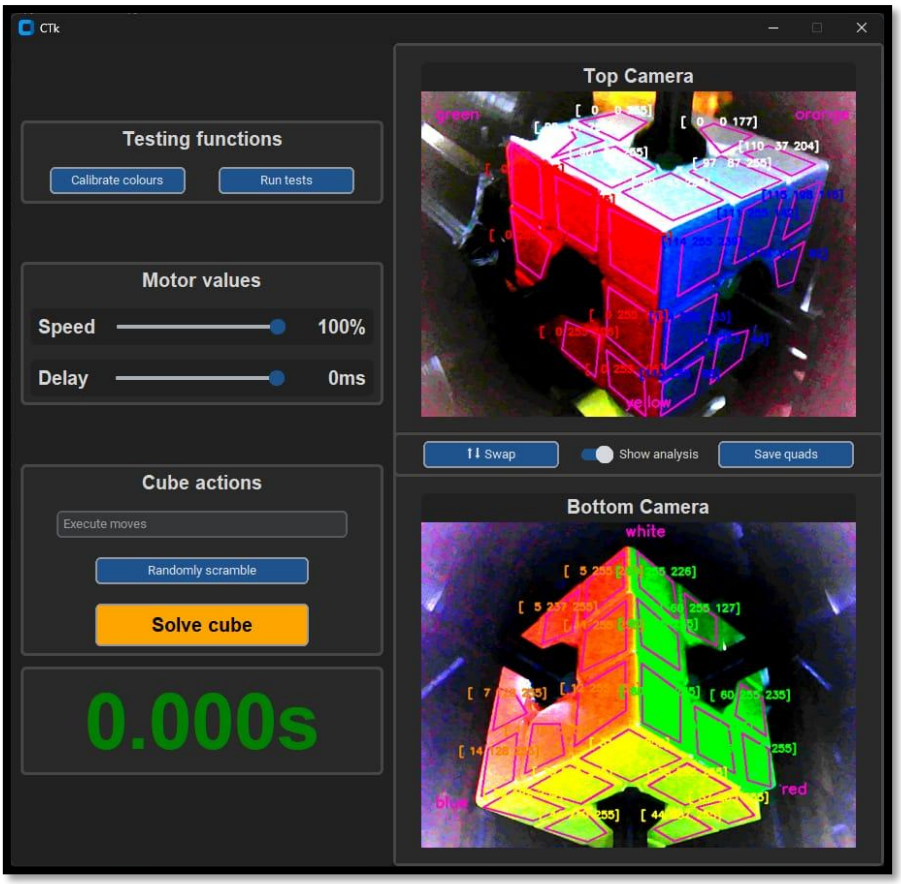


Figure 5-10. Python GUI

The right-hand region shows the live feeds from the top and bottom cameras, with controls for swapping the two cameras, toggling a colour analysis overlay (which displays the detected facelet colours and quadrilateral boundaries directly onto the camera feeds), and saving any adjustments made to the vertex positions.

The left-hand region contains the cube control functions, organised into three groups. The Testing Functions group provides access to the colour calibration routine and the automated test procedure. The Motor Values group has two sliders that control the inter-step delay and inter-move delay parameters. The Cube Actions group has an entry field for issuing arbitrary move sequences manually, a button for performing a random scramble (using WCA-compliant scrambles generated by cubescrambler [20]), and a button for starting a solve. A large solve timer is displayed beneath these controls.

Camera frame loop. Each camera has its own frame update loop, scheduled using the CustomTkinter self.after() method to grab a fresh frame at a configurable interval. When the colour analysis overlay is enabled, the frame is annotated with the quadrilateral outlines and the detected HSV values for each facelet before being displayed. When disabled the raw frame is unmodified apart from some scaling. The two cameras run on independent update loops so each updates as quickly as its own frame rate allows.

Direct command entry. The Cube Actions group's text entry field accepts arbitrary move sequences and forwards them directly to the Arduino. Sequences entered here aren't validated against the current cube state and don't invoke the solver. This was useful during development, both for testing individual motors and for issuing the special two-integer commands that update the firmware's step delay and inter-move delay.

Colour calibration. The calibration routine executes a series of random moves, samples the colour of each facelet within each quadrilateral after every move, and aggregates the samples to produce a set of HSV hue ranges that can reliably distinguish the six colours under the current lighting. A virtual representation of the cube is maintained throughout this process using magiccube [19], which tracks which physical facelet ends up in which position after each move so that samples can be correctly aggregated by colour.

Automated test procedure. The test procedure runs a configurable number of consecutive scramble-and-solve cycles, recording the success or failure of each attempt along with timing data. This lets the reliability and performance of the system to be measured systematically and provides the data presented in Chapter 6.

5.7 Automated Test Procedure

The automatic test GUI *App.tester()* (Appendix A) function performs a configurable number of consecutive scramble-and-solve cycles, with each cycle consisting of a random scramble executed on the physical robot followed by an attempt to detect, solve, and execute a solution. The scrambles are generated using cubescrambler [20], which produces sequences conforming to the WCA regulations for competition scrambles. For each cycle, the function records the success or failure of the attempt, the time spent in colour detection and solver invocation, and the time taken to execute the solve.

6 Testing and Validation

This chapter documents the automated testing performed on the completed system and presents the results.

6.1 Testing Methodology

Each test run consists of a sequence of independent scramble-and-solve cycles. For each cycle it generates a WCA-compliant random scramble using cubescrambler [20], executes it on the physical robot, captures fresh images from both cameras, and attempts to construct a valid cubestring. If the resulting cubestring fails the solver’s validity check, the single-error

correction routine described in Section 5.5 is applied. If the cubestring still cannot be solved – it captures new images from both cameras and repeats the process, up to a maximum of 200 attempts. If a solution is found within the attempt limit, the move sequence is sent to the Arduino and the solve time is recorded. If no solution is found within 200 attempts, the cycle is recorded as a failure.

A "failure" in this testing context means that the colour detection stage couldn't produce a recoverable cubestring within 200 fresh image captures, not that the robot crashed or experienced a mechanical or software fault. Each cycle records the scramble sequence, the solution sequence, success/failure, the total solve time, the number of detection attempts required, the move count produced by the solver, the equivalent total turn count (where 180° moves are counted as two turns), and the resulting turns per second.

6.2 Test Configuration

Two formal test runs of 100 cycles each were performed. Both runs used identical conditions, with the system at its highest reliable operating speed: 100% speed (320µs step delay) and a move delay of 0ms. These values were arrived at through informal testing before the formal runs. At step delays shorter than 320µs, the motors occasionally lose synchronisation under load and produce inaccurate rotations. The 0ms inter-move delay caused no observable instability and so was kept for maximum speed. The system drew approximately 3A on average from the 14V supply during these runs. The 4.8A current limit on the bench supply was not approached at any point.

6.3 Results

Both tests achieved a 100% success rate over their 100 cycles, with no failures observed in either run. Aggregated statistics are shown below:

Metric	Test Run 1 (Appendix K)	Test Run 2
Cycles completed	100	100
Successful solves	100	100
Success rate	100 %	100 %
Mean solve time (s)	2.117	2.146
Fastest solve time (s)	1.500	1.781
Slowest solve time (s)	4.250	5.343
Mean move count	19.68	19.93
Mean turn count	28.66	28.95
Mean turns per second	15.35	15.35
Mean detection attempts	2.94	4.37

First-attempt successes	70	68

Table 1. Aggregated test results

The two runs produced very similar results across all metrics, giving confidence that the figures observed are representative of the system's normal behaviour rather than artefacts of a single favourable run.

Solve times. The mean solve time across both runs is just above 2.1 seconds, with the fastest single solve at 1.5 seconds. For comparison, the current human world record for solving a 3×3×3 Rubik's Cube is 2.76 seconds, set by Teodor Zajder in 2026; the system therefore comfortably beats human capability under typical conditions. The slowest solves in each run (4.250 seconds and 5.343 seconds) correspond to cycles where colour detection needed a lot of fresh image captures before producing a valid cubestring, not to genuinely slow physical execution.

Move efficiency. The Kociemba two-phase algorithm consistently produces near-optimal solutions, with mean move counts of 19.68 and 19.93 across the two runs and a maximum of 22 in any single solve. This is close to the theoretical upper bound of 20 moves for any scramble [22], recognising that the solver is given a finite 0.1 second budget per attempt and so doesn't always reach the optimum. The mean turn count is around 28.8, reflecting that about 9 of the average 19.8 moves in each solution are 180° rotations. The fastest solve at 1.5 seconds corresponded to a solution with a particularly low turn count.

Mechanical performance. The mean turns-per-second figure of 15.35 was identical across both runs, showing that the motor control subsystem performs deterministically. A turn rate of 15.35 TPS corresponds to a per-turn duration of about 65 ms, which matches the expected value derived from the firmware configuration: at a step delay of 320 µs, each step pulse takes 640 µs, and 100 steps are required per 90° rotation, giving a calculated 64 ms per turn. The small discrepancy is attributable to per-move overhead in the firmware loop. No mechanical failures (jams, missed steps, or coupler slippage) were observed at any point during either run.

6.4 Time Budget Breakdown

To understand where time is spent during a typical solve, the contributions of each stage were estimated from the test data. With a mean solve time of around 2.13 s and a mean turn count of 28.8 at 15.35 TPS, motor execution accounts for around 1.88 s of each cycle. The remaining 0.25 seconds is taken up by colour detection and solver invocation. Colour detection alone takes up the majority of that time. The system spends around 90% of each cycle executing physical moves and around 10% on all software processing combined. This breakdown was

consistent across both runs and confirms that further reductions in solve time will require improvements to the mechanical execution stage rather than to the software pipeline.

6.5 Detection Reliability

The number of detection attempts required per solve is the most variable metric and is worth examining more closely. The distribution of attempts per cycle across both runs is shown:

Attempts required	Run 1	Run 2
1 (first-attempt success)	70	68
2 to 5	22	20
6 to 20	5	8
21 to 50	1	2
Greater than 50	1	2
Maximum observed	75	114

Table 2. Distribution of attempts per cycle per test run

In both runs, around 70 of 100 cycles were solved on the first attempt, meaning the colour detection produced a directly valid cubestring (or one repairable by single-error correction) in the majority of cases. A further 20-22 cycles were resolved within five attempts, and only a small number of outliers required substantially more. The maximum observed attempt count was 114, well within the 200-attempt limit; no cycle in either run reached the failure threshold.

The cycles that required many attempts correspond to scrambles in which one or more facelets produce HSV values close to the boundary between two adjacent colour classes – typically between red and orange, between yellow and green, or between white and blue. The small frame-to-frame variation in HSV values from the cameras' sensor noise causes these facelets to flicker between two classifications across successive samples. The detection strategy relies on this flicker eventually resolving to a sample in which all flickering facelets simultaneously land on classifications that produce a valid cubestring. This approach is not fool-proof but significantly increases the success rate on borderline cases.

6.6 Reliability Estimate

The 200 cycles documented in this chapter all succeeded, which corresponds to an observed success rate of 100% within the formal testing. However, some informal testing identified specific scramble configurations that the system can't solve – typically those with multiple borderline facelets simultaneously, exceeding the recoverable error count even after many attempts. Based on this informal testing, the overall reliability is conservatively estimated at

approximately 99%, with the remaining ~1% failure rate attributable entirely to the colour detection stage rather than to mechanical or software faults.

6.7 Limitations Identified

Lighting sensitivity. The colour detection pipeline depends heavily on the lighting environment. Although the supplementary LEDs provide stable, uniform illumination under normal conditions, any change to the ambient lighting, i.e. a curtain being drawn, a cloud passing the window, or a light being switched on, measurably shifts the HSV values of all facelets. This is a fundamental limitation of any HSV-thresholded colour detection approach.

Inability to verify executed moves. The Arduino firmware drives the stepper motors in open loop, with no feedback on whether each step actually produced the corresponding physical rotation. If a motor were to lose synchronisation mid-solve, which is possible at higher speeds, the system would have no way to detect the failure.

Single-error correction limit. The error correction routine can only handle cubestrings with a single misclassification at a time. The small number of scrambles the system can't solve are typically those with multiple borderline facelets that fail to resolve onto a single-error-correctable state across all 200 attempts. A multi-error correction strategy could solve more of these, but at substantially higher computational cost per attempt.

7 Conclusions and Future Work

This final chapter draws together the work documented in the preceding chapters, evaluates the project against its objectives, and identifies the most significant opportunities for future improvement.

7.1 Project Outcome

Measured against the objectives set out in Chapter 1, the project has been a success. A robust mechanical frame holding six stepper motors was designed and 3D-printed; accurate and reliable stepper motor control was implemented in firmware on an Arduino Nano; a computer vision pipeline capable of detecting the colour of each visible facelet and reconstructing the obscured ones was developed; the Kociemba two-phase solving algorithm was integrated and its output translated into executable motor commands; a graphical user interface was built around the system to support operation, calibration, and automated testing; and all of these subsystems were combined into a working autonomous robotic system.

The completed system achieves a mean solve time of approximately 2.13 seconds, with a fastest recorded solve of 1.5 seconds. Across 200 documented test cycles, every cycle was

solved successfully. Informal testing identified a small number of scramble configurations that the system cannot solve, all of which fail at the colour detection stage rather than through any mechanical or software fault; the overall reliability is conservatively estimated at approximately 99%. A user can scramble a cube by hand, insert it into the robot in the marked orientation, press a single button, and have the robot solve it in less than the current human world record time of 2.76 seconds.

7.2 Critical Reflection

While the project achieved its objectives, there's significant scope for improvement in nearly every subsystem. However, many of the improvements identified below were only recognisable in hindsight, problems that could not have been easily anticipated from the outset.

Lessons learned. Several broad observations came out of completing the project. Firstly, iterative prototyping was essential. The motor coupler design went through nine iterations before achieving a satisfactory fit, and the side-mount design had to be reworked entirely after the first single-piece print failed structurally along its layer lines. Neither problem was foreseeable from CAD alone. Secondly, empirical tuning was unavoidable in several places. The HSV classification values, the maximum reliable step rate, and the inter-move delay were all arrived at through testing rather than from first principles. Thirdly, using an established open-source library for the cube-solving algorithm yielded much more value than writing a custom implementation would have. Using the existing Python implementation let the project focus on integration challenges rather than algorithm development. Finally, splitting the work between a host computer and a microcontroller proved correct in retrospect: each device handles the work it's best suited to.

Project management. With more formal project planning at the start progress would likely have been smoother. In practice, several stages of the project hit unanticipated obstacles that took more time than initially estimated. The colour detection stage in particular went through multiple iterations: the migration from RGB to HSV, the switch from rectangular sampling regions to perspective-projected quadrilaterals, and the development of the error correction process. Each refinement was driven by empirical observations that only became visible after the previous version was implemented. While many of these unknowns were genuinely difficult to predict, more formal planning would at least have given clearer visibility of where the project stood relative to its overall timeline.

7.3 Future Work

Many opportunities for further development have been identified, both during the build and through later testing.

Mechanical refinements. The mechanical frame is functional but would benefit from a refined design that simplifies cube insertion. The current process is workable but awkward, and it is easy to accidentally shift the cameras during insertion. A revised frame in which the cube can be inserted through an easier process and one in which the cameras are more rigidly fixed to the frame would address both issues. Moving the entire assembly to a more permanent enclosure, ideally with an integrated LCD display and physical button controls in addition to the existing software interface, would make the system more presentable and easier to operate without a connected computer.

Motor and driver improvements. The choice of stepper motors was deliberate and appropriate for a first-generation system, but the per-turn duration is the dominant component of the total solve time. Replacing the steppers with brushless DC motors with encoders, would allow significantly higher rotation speeds at the cost of much more control complexity. As an interim improvement, implementing acceleration ramping in the firmware would allow higher peak step rates than the current constant-speed profile without requiring different motor hardware. Activating each driver one-by-one, rather than all at once, would reduce the inrush current peak and allow a lower, safer current limit.

Power supply. The current dependency on a bulky bench power supply and a separate battery pack is a practical inconvenience and limits portability. A purpose-selected fixed-voltage 14V supply with a sufficient current, combined with a properly working 6V rail derived from the same supply using a buck converter, would replace both the bench supply and the battery pack with a single neat solution.

Custom electronics. The current breadboard / protoboard assembly is appropriate for prototyping but is fragile and visually untidy. A custom PCB hosting the Arduino, the six stepper motor drivers, the supplementary lighting connections, and the buck converter would substantially reduce the wiring footprint, improve electrical reliability, and produce a more professional finished system.

Migration to a Raspberry Pi. The host software is currently run on a Windows PC or laptop. Moving it to a Raspberry Pi was attempted during the project but was not completed: the cameras and the Pi together drew enough current that the Pi would intermittently shut down, and additional integration issues with the UI and camera handling under the Pi's environment were not resolved within the project timeline. A future revision could pursue this with an appropriate power supply and time set aside for the necessary debugging. A successful Pi deployment would let the system run as a self-contained appliance rather than depending on a connected PC.

Vision system. The most significant improvement opportunity in the entire system is the vision subsystem. The chosen Trust Spotlight Pro webcams are inexpensive but produce relatively dim, low-exposure images that need aggressive contrast normalisation in software. Higher-resolution cameras with greater dynamic range, faster frame rates, and better low-light performance would substantially reduce noise. The supplementary lighting could also be redesigned to produce more uniform illumination across all visible facelets. Independent HSV ranges per facelet, rather than the global ranges currently in use, would directly address the issue of facelets at different camera angles producing systematically different HSV values for the same physical colour. Finally, the calibration and quadrilateral placement steps could be automated entirely through image analysis by analysing the cube boundaries, detecting the individual facelet sampling regions, and deriving the colour ranges automatically

Software refinements. The host-side codebase reached a working state but would benefit from substantial refactoring. The user interface in particular has grown organically and would be cleaner if redesigned around a more structured architecture. Several features remain partially implemented or absent: writing the calibration output directly to the JSON configuration file rather than reporting it to the operator, displaying the move count alongside the solve timer, and exposing additional diagnostic information during automated test runs. A multi-error correction strategy could help solve the small number of scrambles that currently exceed the system's recovery capability.

7.4 Closing Remarks

The project's value lies primarily in its educational and demonstrative dimensions. As a personal undertaking, it required integrating mechanical design, electronics, embedded firmware, computer vision, and host-side software into a single coherent system, and produced substantial learning across all of these areas. As a finished robot, the system is a tangible demonstration of what's achievable with accessible hardware and freely available software libraries, and provides a platform on which any of the future improvements identified above could be explored. While the robot isn't, in any practical sense, a useful product to the wider world, projects like this have continuing value as platforms for engineering education and as accessible demonstrations of integrated robotics for outreach to students considering technical disciplines.

The completed system has been built, tested, and shown to perform reliably against its objectives. The remaining opportunities for improvement are extensive but well-understood, and stand as a record of the engineering judgement gained through building the system in the first place.

8 References

[1] D. Dorran, “Report Template and Guide to Writing Style”. Accessed on: Jan. 25, 2020. [Online]. Available: http://eleceng.tudublin.ie/dorran/report_info/. [Accessed: Apr-2026].

[2] M. Farrell, “Masters in Advanced Engineering Project Marking Scheme and Guidelines”, Dublin Institute of Technology, Dublin, Ireland, March 2008.

[3] “Kociemba’s Algorithm,” SpeedSolving Wiki. [Online]. Available: https://www.speedsolving.com/wiki/index.php/Kociemba%27s_Algorithm. [Accessed: Apr-2026].

[4] H. Kociemba, “RubiksCube-TwophaseSolver,” GitHub. [Online]. Available: https://github.com/hkociemba/RubiksCube-TwophaseSolver. [Accessed: Apr-2026].

[5] Stevan Kostoski, “From zero to Rubik’s Cube solving robot,” Medium. [Online]. Available: https://stevcooo.medium.com/from-zero-to-rubiks-cube-solving-robot-ad3a2838cd. [Accessed: Apr-2026].

[6] Aaed Musa, “Rubik’s Cube Solver,” Hackaday.io. [Online]. Available: https://hackaday.io/project/190071-rubiks-cube-solver. [Accessed: Apr-2026].

[7] Aaed Musa, “I Built a Rubik’s Cube Solver,” YouTube, Jan. 7, 2022. [Video]. Available: https://youtu.be/V8gHTKWw--Y. [Accessed: Apr-2026].

[8] Jay Flatland, “World’s Fastest Rubik’s Cube Solving Robot,” YouTube, Nov. 15, 2023. [Video]. Available: https://youtu.be/ixTddQQ2Hs4. [Accessed: Apr-2026].

[9] “CUBOTino, the Rubik’s Cube solving robot,” Raspberry Pi, [Online]. Available: https://www.raspberrypi.com/news/cubotino-the-rubiks-cube-solving-robot/. [Accessed: Apr-2026].

[10] HiDe Tech, “dual Axis Rubik’s cube solving robot (Arduino/Processing),” YouTube, Jan. 12, 2021. [Video]. Available: https://www.youtube.com/watch?v=o-auWQzWKxM. [Accessed: Apr-2026].

[11] Aaed Musa, “I Built a Rubik’s Cube Solving Robot,” YouTube, Dec. 1, 2023. [Video]. Available: https://www.youtube.com/watch?v=m0bMMALYMYk. [Accessed: Apr-2026].

[12] Handsontec, “17HS4401S 1.7A Torque: 43N.cm Stepper Motor,” Datasheet. [Online]. Available: https://www.handsontec.com/dataspecs/17HS4401S.pdf. [Accessed: May-2026].

[13] Trinamic Motion Control GmbH & Co. KG, “TMC2202, TMC2208, TMC2224 Datasheet,” Rev. 1.14, Feb. 2023. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/TMC2202_TMC2208_TMC2224_datasheet_rev1.14.pdf. [Accessed:May-2026].

[14] Allegro MicroSystems, “A4988: DMOS Microstepping Driver with Translator and Overcurrent Protection,” Datasheet. [Online]. Available: https://www.pololu.com/file/0j450/a4988_dmos_microstepping_driver_with_translator.pdf. [Accessed: May-2026].

[15] Arduino, “Arduino Nano.” [Online]. Available: https://docs.arduino.cc/hardware/nano/. [Accessed: May-2026].

[16] OpenCV, “opencv-python,” GitHub. [Online]. Available: https://github.com/opencv/opencv-python. [Accessed: May-2026].

[17] T. Schimansky, “CustomTkinter.” [Online]. Available: https://customtkinter.tomschimansky.com. [Accessed: May-2026].

[18] C. Liechti, “pyserial,” GitHub. [Online]. Available: https://github.com/pyserial/pyserial. [Accessed: May-2026].

[19] T. Rinca, “magiccube,” GitHub. [Online]. Available: https://github.com/trincaog/magiccube. [Accessed: May-2026].

[20] koma52, “cubescrambler,” GitHub. [Online]. Available: https://github.com/koma52/cubescrambler. [Accessed: May-2026].

[21] D. Singmaster, *Notes on Rubik’s Magic Cube*. NJ, USA: Enslow Publishers, 1981.

[22] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “The Diameter of the Rubik’s Cube Group is Twenty,” *SIAM Journal on Discrete Mathematics*, vol. 27, no. 2, pp. 1082–1105, 2013.

[23] World Cube Association, “WCA Official Results – Records.” [Online]. Available: https://www.worldcubeassociation.org/results/records?show=mixed. [Accessed: May-2026].

[24] Engineers Ireland, “Code of Ethics.” [Online]. Available: https://www.engineersireland.ie/Professionals/News-Insights/Resource-centre/Member-resources/Code-of-ethics. [Accessed: May-2026].

[25] United Nations, “Goal 4: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.” [Online]. Available: https://sdgs.un.org/goals/goal4. [Accessed: May-2026].

9 Appendices

Appendix A. Project Repository and Demo Video

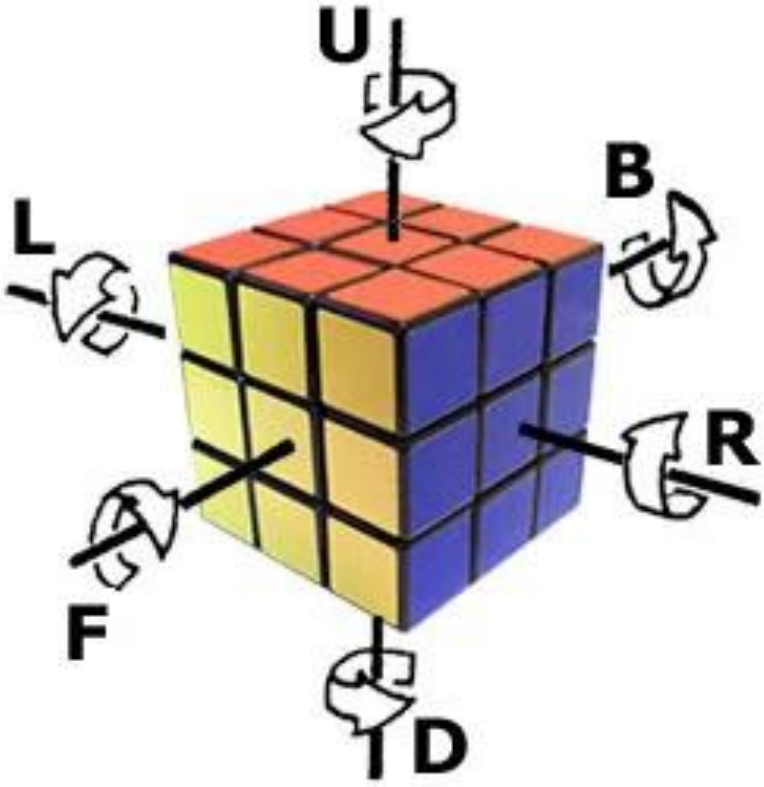
The complete source code, 3D model files, test data, and supporting documentation for this project are hosted publicly on GitHub: https://github.com/PeterMyler/CubeRobot

The Python source code consists of over 1,100 lines so not all of it is included in the appendix.

A demo video of the working project can be viewed here: https://youtu.be/nHWHOQfeOHc

Appendix B. Singmaster Notation

Singmaster notation is the universal standard language used to write sequences of moves (algorithms) on a Rubik's Cube.



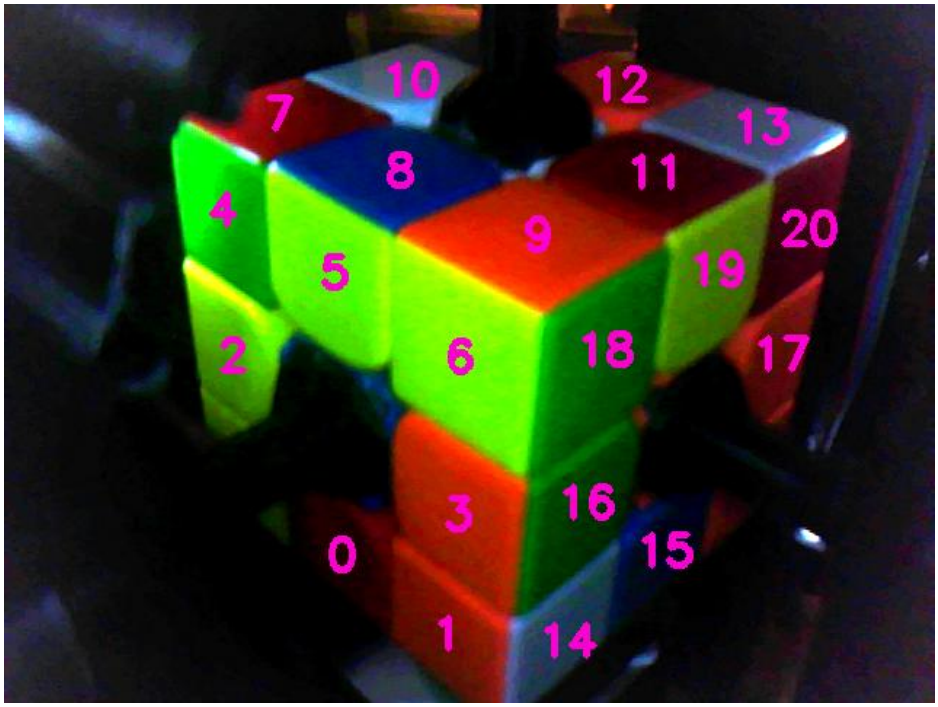
Appendix C. Bill of Materials

System Component	Cost
6x NEMA 17HS4401 Stepper Motors	€60
6x TMC2208 Stepper Motor Drivers	€20
2x Trust Light Pro USB Webcams	€15
Arduino Nano	previously purchased
4x white LEDs	previously purchased
4xAA batteries & battery holder	previously purchased
MoYu RS3M 2020 Speed Cube	previously purchased (€15)
NANKADF WPS3010H variable bench power supply	previously purchased (€60)

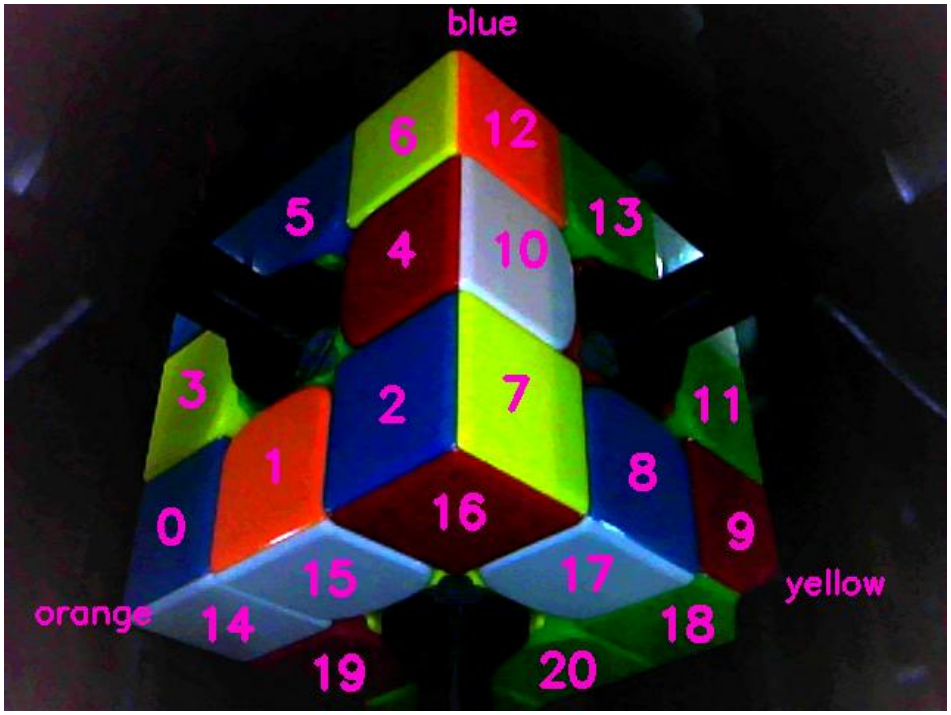
Extra Materials and Consumables:

- 3D printer filament (~300g)
- M3 nuts and bolts
- Wires and connectors
- Protoboards and breadboard
- Solder and flux
- Lots of time (€∞)

Appendix D. Top camera facelet order



Appendix E. Bottom camera facelet order



Appendix F. TwophaseSolver Cube Notation

RubiksCube-TwophaseSolver / enums.py

Code Blame 157 lines (144 loc) · 3.54 KB

```
class Facelet(IntEnum):
    """
    The names of the facelet positions of the cube
                 |************|
                 |*U1**U2**U3*|
                 |************|
                 |*U4**U5**U6*|
                 |************|
                 |*U7**U8**U9*|
                 |************|
     |************|************|************|************|
     |*L1**L2**L3*|*F1**F2**F3*|*R1**R2**R3*|*B1**B2**B3*|
     |************|************|************|************|
     |*L4**L5**L6*|*F4**F5**F6*|*R4**R5**R6*|*B4**B5**B6*|
     |************|************|************|************|
     |*L7**L8**L9*|*F7**F8**F9*|*R7**R8**R9*|*B7**B8**B9*|
     |************|************|************|************|
                 |************|
                 |*D1**D2**D3*|
                 |************|
                 |*D4**D5**D6*|
                 |************|
                 |*D7**D8**D9*|
                 |************|
    A cube definition string "UBL..." means for example: In position U1 we have the U-color, in position U2 we have the
    B-color, in position U3 we have the L color etc. according to the order U1, U2, U3, U4, U5, U6, U7, U8, U9, R1, R2,
    R3, R4, R5, R6, R7, R8, R9, F1, F2, F3, F4, F5, F6, F7, F8, F9, D1, D2, D3, D4, D5, D6, D7, D8, D9, L1, L2, L3, L4,
    L5, L6, L7, L8, L9, B1, B2, B3, B4, B5, B6, B7, B8, B9 of the enum constants.
    """
```

Appendix G. Cube data json file


```
{
  "camera_ids": [1, 0],
  "motor_speed": 320,
  "motor_delay": 0,
  "bottom_camera_boxes": [[[90, 337], [133, 313], [127, 383], [80, 407]], [[153, 298], [202, 292], [201, 342], [147, 364]], [[232, 245], [300, 205], [297, 292], [225, 330]], [[109, 241], [131, 248], [149, 277], [99, 303]], [[238, 161], [292, 120], [290, 181], [231, 220]], [[172, 138], [219, 100], [210, 161], [166, 178]], [[236, 87], [290, 43], [289, 94], [233, 139]], [[318, 210], [375, 252], [378, 324], [322, 288]], [[397, 265], [447, 294], [456, 364], [402, 335]], [[463, 313], [502, 336], [497, 372], [471, 382]], [[313, 118], [373, 171], [376, 224], [320, 187]], [[459, 232], [491, 255], [496, 309], [451, 278]], [[302, 42], [355, 88], [358, 134], [306, 95]], [[378, 103], [425, 139], [425, 181], [382, 159]], [[138, 404], [198, 421], [157, 442], [102, 422]], [[214, 360], [282, 387], [235, 415], [157, 392]], [[301, 311], [362, 345], [300, 374], [231, 347]], [[394, 360], [442, 385], [384, 408], [334, 388]], [[464, 399], [484, 421], [434, 441], [395, 420]], [[235, 431], [262, 455], [235, 465], [181, 451]], [[340, 450], [378, 429], [414, 451], [359, 469]]],
  "top_camera_boxes": [[[213, 339], [262, 359], [256, 421], [207, 380]], [[279, 384], [335, 428], [337, 479], [284, 437]], [[135, 185], [191, 224], [175, 275], [142, 250]], [[280, 279], [350, 326], [342, 406], [277, 351]], [[142, 160], [155, 102], [181, 113], [184, 190]], [[197, 125], [260, 158], [257, 235], [202, 201]], [[276, 173], [355, 211], [345, 298], [278, 255]], [[225, 63], [215, 46], [266, 67], [190, 92]], [[282, 77], [339, 110], [277, 138], [201, 97]], [[365, 111], [442, 151], [370, 180], [293, 139]], [[236, 42], [299, 28], [336, 43], [295, 70]], [[442, 90], [531, 126], [472, 152], [396, 113]], [[425, 35], [482, 53], [436, 78], [405, 43]], [[515, 75], [581, 100], [541, 114], [478, 86]], [[355, 431], [415, 391], [402, 445], [352, 479]], [[435, 360], [481, 352], [470, 391], [425, 431]], [[367, 327], [437, 287], [424, 353], [360, 399]], [[520, 248], [564, 221], [547, 280], [525, 288]], [[385, 214], [455, 184], [437, 267], [377, 302]], [[475, 174], [535, 142], [510, 228], [460, 254]], [[549, 138], [585, 122], [569, 182], [529, 208]]],
  "hsv_ranges": []
}
```

Appendix H. Camera.py

This appendix contains Python code from the Camera.py script.

Appendix H-1. Camera class init

```python
# camera variables
CAMERA_RESOLUTION = (640, 480)
CAMERA_NAME = "Trust Webcam"
CAMERA_EXPOSURE = -1
DESIRED_CONTRAST = 2000
```

```python
class Camera:  6 usages  PeterTheCool
    def __init__(self, device_id, flip_upsidedown=False, name=None, box_coords=None):
        self.device_id = device_id
        self.flip_upsidedown = flip_upsidedown
        self.name = name
        self.label = None  # label from GUI script
        self.hidden_corner_indexes = None
        self.hidden_corner_text_coords = None
        self.width, self.height = CAMERA_RESOLUTION
        self.box_coords = box_coords

        if not(len(devices) > device_id and devices[device_id] == CAMERA_NAME):
            self.cap = None
            return

        try:
            # Connect to camera:
            self.cap = cv2.VideoCapture(device_id, cv2.CAP_DSHOW)

            # Apply camera settings:
            # set camera resolution
            self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, self.width)
            self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, self.height)
            # set exposure
            self.cap.set(cv2.CAP_PROP_EXPOSURE, CAMERA_EXPOSURE)
            self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 0)

            print("Connected to camera", device_id)
        except Exception as e:
            self.cap = None
            print(e)
```

Appendix H-2. Camera frame handling

```
107         def get_frame(self):  11 usages  PeterTheCool
108             if not self.cap: return None
109             ret, frame = self.cap.read()
110 
111             if not ret or frame is None:
112                 return None
113 
114             # flip image 180 degrees
115             if self.flip_upsidedown:
116                 cv2.rotate(frame, cv2.ROTATE_180, frame)
117 
118             # convert to RGB
119             frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
120 
121             # increase image contrast
122             cv2.normalize(frame, frame, 0, DESIRED_CONTRAST, cv2.NORM_MINMAX)
123 
124             return frame
```

Appendix H-3. HSV colour classification values

```
6   #                0      1        2        3       4       5
7   COLOUR_NAMES = ("red", "orange", "yellow", "green", "blue", "white")
8   RGB_COLOUR_VALUES = ((255, 0, 0), (255, 128, 0), (255, 255, 0), (0, 255, 0), (0, 0, 255), (255, 255, 255))  # ideal values
9   COLOUR_HUE_RANGES = (0, 3, 23, 45, 84, 146, 181)  # (0, 3, 25, 53, 70, 160, 181)
10  WHITE_LIMITS = (60, 185, 120, 75, 130)  # low max s, [max s, min v, min h, max, h]
11  BAD_RED_LIMITS = (8, 130)  # max hue, max value
12  BAD_GREEN_LIMITS = (40, 175)  # min hue, max v - green for Value below threshold, yellow for above
13  BOX_COLOUR = (255, 0, 210)
```

Appendix H-4. HSV colour classification function

```
28  def classify_hsv_colour(hsv_colour):  1 usage  PeterTheCool
29      h, s, v = hsv_colour
30 
31      # white - low Saturation and high Value
32      if (s <= WHITE_LIMITS[0]) or (s <= WHITE_LIMITS[1] and v >= WHITE_LIMITS[2]
33                                    and WHITE_LIMITS[3] <= h <= WHITE_LIMITS[4]):
34          return 5
35 
36      # special case to determine red or orange for low hue
37      if COLOUR_HUE_RANGES[1] <= h <= BAD_RED_LIMITS[0]:
38          # red for Value below v, orange for above
39          return 0 if v <= BAD_RED_LIMITS[1] else 1
40 
41      # special case to determine green or yellow
42      if BAD_GREEN_LIMITS[0] <= h <= COLOUR_HUE_RANGES[3]:
43          # green for Value below threshold, yellow for above
44          return 3 if v <= BAD_GREEN_LIMITS[1] else 2
45 
46      # determine colour (match to colour hue ranges)
47      for i in range(len(COLOUR_HUE_RANGES) - 1):
48          if COLOUR_HUE_RANGES[i] <= h < COLOUR_HUE_RANGES[i + 1]:
49              return i % 5  # wrap back around to red
50 
51      return None
```

Appendix H-5. Hidden corner piece colours

```
# rubiks corner-piece colours in order of top, right, front
CORNER_COLOURS = ((5, 4, 0), (5, 0, 3), (5, 3, 1), (5, 1, 4), (2, 4, 1), (2, 1, 3), (2, 3, 0), (2, 0, 4))
```

```
def get_hidden_corner_colour(colour1, colour2):  3 usages  PeterTheCool
    # find matching piece with matching colours in same order
    correct_order = (colour1, colour2)
    for corner in CORNER_COLOURS:
        for i in range(3):
            # if the current corner and the next one (wrapped around)
            # equal the correct order -> return the corner after that
            if (corner[i], corner[(i + 1) % 3]) == correct_order:
                return corner[(i + 2) % 3]
    return None
```

Appendix H-6. Calculating median HSV values

```
def get_median_hsv_colours(self, img):  3 usages  PeterTheCool *
    # get the median HSV in each given pixel coord
    colours = []
    hsv_img = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)

    for quad in self.box_coords:
        pts = np.array(quad)
        # get the bounding box of the quad
        x, y, w, h = cv2.boundingRect(pts)
        area = hsv_img[y:y + h, x:x + w]
        # area = cv2.GaussianBlur(area, (5, 5), 0)

        # shift points to new area space coords
        pts_shifted = pts - [x, y]

        # mask quad area
        mask = np.zeros((h, w), dtype=np.uint8)
        cv2.fillPoly(mask, [pts_shifted], 255)

        # get pixels with mask applied
        pixels = area[mask == 255]

        if len(pixels) == 0:
            print("Median calculation failed")
            colours.append(None)
        else:
            # append median hsv values
            colours.append(np.median(pixels, axis=0).astype(int))

    return colours
```

Appendix I. Cube.py

This appendix contains Python code from the Cube.py script.

Appendix I-1. Cube class init

```
# arduino config constants
ARDUINO_PORT = "COM9"
ARDUINO_BAUD_RATE = 115200
```

```
def connect_to_arduino():  1 usage  PeterTheCool
    print(end="Connecting to arduino... ")

    try:
        ard = serial.Serial(port=ARDUINO_PORT, baudrate=ARDUINO_BAUD_RATE, timeout=1, write_timeout=None)
    except serial.serialutil.SerialException as e:
        # couldn't connect to arduino
        print(" Failed:", e)
        return None

    # wait for arduino to be ready
    while not ard.readline(): pass
    print("Connected")
    return ard
```

```
class Cube:  1 usage  PeterTheCool
    def __init__(self):  PeterTheCool
        self.state = None
        self.arduino = connect_to_arduino()

        if self.arduino is not None:
            sleep(0.5)
            self.arduinoWriteRead("U U U U")
```

Appendix I-2. Camera to cube state conversion

```
# arrays to convert camera colour values to cube state colours
camB_conv = (42, 43, 44, 39, 41, 37, 38,
             24, 25, 26, 21, 23, 18, 19,
             33, 30, 27, 28, 29, 34, 32)
camB_hidden_conv = (15, 6, 53)
camT_conv = (16, 17, 12, 14, 9, 10, 11,
             8, 5, 2, 7, 1, 3, 0,
             51, 52, 48, 50, 45, 46, 47)
camT_hidden_conv = (20, 36, 35)
centres_conv = (49, 22, 31, 13, 40, 4)
```

```python
def convert_cam_data_to_cubestate(camB_c, camB_hc, camT_c, camT_hc):  1 usage  PeterTheCool
    cubestate = [""] * 54
    # map colour data from camera to cube string indexes
    for col, conv in zip(camB_c, camB_conv):
        cubestate[conv] = " " if col is None else colours_map[col]
    for col, conv in zip(camB_hc, camB_hidden_conv):
        cubestate[conv] = " " if col is None else colours_map[col]
    for col, conv in zip(camT_c, camT_conv):
        cubestate[conv] = " " if col is None else colours_map[col]
    for col, conv in zip(camT_hc, camT_hidden_conv):
        cubestate[conv] = " " if col is None else colours_map[col]
    # map centres
    for col, conv in enumerate(centres_conv):
        cubestate[conv] = colours_map[col]

    return "".join(cubestate)
```

Appendix I-3. Arduino serial communication

```python
    def arduinoWriteRead(self, command):  5 usages  PeterTheCool
        if self.arduino is None: return None

        # sends data to arduino, waits for response, and then returns it
        self.arduino.write(bytes(command.upper(), 'utf-8'))
        while not (data := self.arduino.readline()): pass
        return data.decode('utf-8').strip()
```

Appendix I-4. Solver call

```python
    def solve_cube(self):  2 usages  PeterTheCool *
        # attempt to solve
        solve = sv.solve(self.state, 0, 0.1)
        if not solve.startswith("Error"):
            # solve success
            return twophaseToNormal(solve)
```

Appendix I-5. Error correction

```python
# for error correction
BOTTOM_HIDDEN_CORNERS = ((18, 9), (12, 6), (0, 14))
TOP_HIDDEN_CORNERS = ((7, 4), (20, 13), (1, 14))
ERROR_COLOURS = (1, 0, 3, 2, 5, 4)
```

```
    def solve_cube(self):  2 usages  PeterTheCool *
        # attempt to solve
        solve = sv.solve(self.state, 0, 0.1)
        if not solve.startswith("Error"):
            # solve success
            return twophaseToNormal(solve)

        # attempt to fix cube by swapping each colour with its possible error
        old_cubestate = self.state
        for tp_ind in range(len(old_cubestate)):
            # skip hidden corner colours
            if tp_ind in camB_hidden_conv or tp_ind in camT_hidden_conv: continue

            curr_state = list(old_cubestate)
            old_col = colours_map.index(old_cubestate[tp_ind])
            new_col = old_col + (1 if old_col%2==0 else -1)  # ERROR_COLOURS[old_col]
            # swap colour
            curr_state[tp_ind] = colours_map[new_col]

            # convert twophase index to camera index
            cam_ind = hidden_corner_indexes = conv = hidden_conv = None
            if tp_ind in camB_conv:
                cam_ind = camB_conv.index(tp_ind)
                hidden_corner_indexes = BOTTOM_HIDDEN_CORNERS
                conv = camB_conv
                hidden_conv = camB_hidden_conv
            elif tp_ind in camT_conv:
                cam_ind = camT_conv.index(tp_ind)
                hidden_corner_indexes = TOP_HIDDEN_CORNERS
                conv = camT_conv
                hidden_conv = camT_hidden_conv

            # if used in a hidden corner calc - redo it
            if not cam_ind is None:
                for h_ind, corner in enumerate(hidden_corner_indexes):
                    if cam_ind in corner:
                        cols = [None, None]
                        cor_ind = corner.index(cam_ind)
                        cols[cor_ind] = new_col
                        cols[cor_ind - 1] = colours_map.index(curr_state[conv[corner[cor_ind - 1]]])

                        corner_col = get_hidden_corner_colour(cols[0], cols[1])
                        if corner_col is not None:
                            curr_state[hidden_conv[h_ind]] = colours_map[corner_col]
                            # print("Fixed hidden corner:", self.state)
                            break

            # attempt to solve
            solve = sv.solve("".join(curr_state), 0, 0.1)
            if not solve.startswith("Error"):
                # solve success
                print(solve)
                return twophaseToNormal(solve)

        # send cubestate to solver
        solve = sv.solve(self.state, 0, 0.1)
        return twophaseToNormal(solve)
```

Appendix J. GUI.py

Appendix J-1. Implementing the Camera class

```python
def connect_to_cameras(cube_data, b_label: ctk.CTkLabel, t_label: ctk.CTkLabel):  1 usage  PeterTheCool
    # Bottom camera
    camB = Camera.Camera(cube_data["camera_ids"][0], flip_upsidedown=True, name="Bottom", box_coords=cube_data["bottom_camera_boxes"])
    camB.label = b_label
    camB.hidden_corner_indexes = Cube.BOTTOM_HIDDEN_CORNERS
    camB.hidden_corner_text_coords = ((530, 400), (300, 20), (20, 420))
    # Top camera
    camT = Camera.Camera(cube_data["camera_ids"][1], name="Top", box_coords=cube_data["top_camera_boxes"])
    camT.label = t_label
    camT.hidden_corner_indexes = Cube.TOP_HIDDEN_CORNERS
    camT.hidden_corner_text_coords = ((20, 40), (550, 40), (300, 465))

    return camB, camT

def get_min_and_max_hsv(col):  3 usages  PeterTheCool
    return [(min(col,key=lambda l:l[n])[n], max(col,key=lambda l:l[n])[n]) for n in range(3)]
```

Appendix J-2. Solve handler function

```python
    def solve_cube(self, callback=None):  2 usages  PeterTheCool
        if self.camT.cap is None or self.camB.cap is None:
            print("Cameras not connected")
            if callback is not None: callback(None)
            return None

        # return if ui is disabled
        if self.ui_disabled:
            return None

        # set start time
        self.start_timer()
        # attempt to solve many times
        for attempt in range(200):
            # get colour data from camera frames
            B_data, T_data = self.get_colour_data_from_cams()
            # set cubestate
            self.cube.set_cubestate(B_data[1], B_data[2], T_data[1], T_data[2])
            # try to solve
            solve = self.cube.solve_cube()
            if solve == "":
                print("Cube already solved!")
                self.stop_timer()
                if callback is not None: callback(solve)
                return solve
            elif not solve.startswith("Error"):
                solve_result = f"Attempt {attempt+1}. Solve: {solve}"
                self.send_to_arduino(solve_result, callback=callback)
                return solve

        self.stop_timer()
        print("Couldn't solve cube")
```

Appendix K. Test 1

Tests 15-81 were hidden for the purposes for taking this image. The full data can be found in the Test_1.csv file in the GitHub repository.

Test #	Scramble	Solution	Success	Time	Attempt	Moves	Turns	TPS
0	B2 D B' L2 D B2 U R D L' R2 B' D2 L D' B' R U F' L' R2 F2 D F2 U2	F U2 B L2 D2 F2 L2 D2 B F' L B' D2 U R2 D' L B2 L	TRUE	2.078	1	19	28	15.34
1	B D2 R2 D2 F' D2 R2 L2 D' B' U D F B2 U B2 D' U2 F' D F2 L' F2 B2 D2	U B R2 U' L2 D B U' D' L B D2 F2 L2 F R2 B2 U2 R2 L2	TRUE	2.186	1	20	30	15.35
2	L D2 U R F' U2 D' R D' L' D' L2 U2 B2 R2 L U' F2 R2 L U' L R F' R	R2 U' B2 L2 B2 R2 F2 U' F2 R' U R2 F' D' B2 F D2 L D F2	TRUE	2.188	1	20	31	15.35
3	L U2 D' R2 D U' F2 B2 L' B' R' L2 B2 D' B2 L2 B' L2 R2 D2 U' F B2 R' U	U' L U F2 R' U B D' L' U R' U' R2 D L2 F2 U D L2 U R2	TRUE	1.921	1	21	27	15.36
4	L' U2 F2 R' F2 B' U D' F2 B2 L U L' D2 L2 U2 F' R' U D F' D' U B2 U'	L2 R2 U L2 U' B2 L2 U' L2 U2 F D U2 B' L R F2 L D L' F'	TRUE	2.233	2	21	30	15.35
5	F B D2 R' D2 R' D2 F D L U2 L F' D U' R D' F2 D2 B' D F' R' B U'	R' F2 U F' L2 F2 D L2 U B' L U L2 D B2 D' B2 R2 D' B2	TRUE	2.141	1	20	29	15.35
6	F D' F' D2 B R2 L F D2 F' B D' U2 F' L' B U' F B D2 B2 R' D L2 D'	L F' L2 U' L2 F B' U2 B L' U R2 D2 L2 F2 D' F2 R2 U L2	TRUE	2.188	1	20	30	15.35
7	B D L' D2 F U2 L2 U R2 B F' D B2 F' L' F R' L2 B2 U' B2 R2 F D B2	F U2 F R L2 F' L D' F' U' F2 R' F2 R2 U F2 R2 L2 F2 U	TRUE	2.031	1	20	29	15.35
8	D2 L2 B' L U2 R B2 F2 D2 F R D B2 U2 D2 F2 R2 L' U2 R' D' L B F' R'	R B' R' D L' B' R F' R' F2 U2 B R2 U2 F' L2 F2 D2 B2	TRUE	2.015	1	19	27	15.35
9	F U' B' R U F' U' R2 D B2 D' L2 R D2 U2 R D2 B' U R U2 R2 U' R U2	U R' D' R L' D2 B L' F' U R D' R2 D' L2 U2 R2 D R2 D' F2	TRUE	1.969	1	21	28	15.35
10	F B' L' R2 B U2 B' D R' F' D U L F' B U L2 U R' B' R B F2 R L'	F2 D L2 R2 D R2 D' B2 D U' R U' F R U L' B R U L2 B'	TRUE	2	1	21	27	15.34
11	D2 L2 U' L2 R B2 U' R2 U' B2 L' R B' D B2 F2 U' D L' B' D L' R D B	R2 F2 L D' B U R L F D' R' U2 F' D2 R2 B D2 F B R2	TRUE	1.905	1	20	27	15.36
12	R F U2 F2 D B2 U' L B' R2 F B R2 F2 B2 L2 D2 B F2 U' F' L' U2 F' L2	F' L2 U' B' D2 R' L2 D B2 D R' U2 B' U2 L2 D2 F2 R2 F	TRUE	2.155	5	19	29	15.34
80	F L2 R B U B' R' U2 B D2 F B2 L2 U' R' U' R' D2 R L2 B2 D' B R B	D B' R' F R2 D2 L U' F U L F2 U D F2 B2 U2 L2 U' F2 D'	TRUE	2.031	1	21	29	15.36
81	D' L2 D' U2 B' D B2 L2 B2 R' L' U L2 U2 R U B' R L2 U' B' L F2 R' B'	R B2 L' F2 D' F R2 D F2 D' B' U F2 U F2 B2 D2 R2 D L2	TRUE	2.11	1	20	30	15.35
82	L' F2 L B' U2 D2 L B2 R D B' R' F2 R' D2 F' L R' U2 L' U2 B2 R' L2 F	R2 F' R2 L D2 F2 B D R U' F D L2 B2 D R2 D B2 L2 D R2	TRUE	2.265	4	21	31	15.35
83	U2 R' U' D' F' L2 D' L' R' D B' L2 D U F2 R2 B2 U F U L B2 F U D2	D F R2 B' R2 U L' B R F U' R2 F2 D2 R2 F' D2 B' R2 L2	TRUE	2.141	1	20	29	15.36
84	B' L' B F2 R D2 B2 L2 D' F B2 D' F B2 D' B2 D' R2 U B' L' R' B2 L' R2	F' L2 F' U2 F R2 U D2 F' R2 U B2 L F2 B2 D2 B2 R' L' U2	TRUE	2.296	1	20	31	15.35
85	F2 B' D B2 L2 F' D' F D2 F2 U' R L F B' U2 B2 D2 U B2 U' L D' U2 B'	L' B L' U' B L F2 L2 U R F U2 R2 D2 F2 R' B2 L' U2	TRUE	2	1	19	27	15.35
86	L' U' R2 L B' R2 D2 R2 U' D R2 F2 B2 R L2 B' L' F B U' R2 D2 F' B D2	R U' R' F2 U F' L2 B2 U' F R' U' D' L2 D F2 D2 F2 B2 U' R2	TRUE	2.109	1	21	30	15.35
87	R' L B2 D' R' F R' F' D' U F2 B D F' D' L B' R2 L' B R2 U L' D' B'	F2 D2 R2 U' L F B2 L' F' U2 L U2 B2 R L2 U2 L2 U2	TRUE	2.125	1	18	29	15.34
88	L' F2 U2 D' L2 D' B F' L' F' B' R' B R' U B' D2 L' D L R B2 L2 U' L	F R D' L B2 R D2 B D F2 U R B' L2 B L2 B2 U2 D2 R2 D2 L2	TRUE	2.312	1	22	33	15.35
89	D' L' F U' F' L2 U' F2 L R' D' R F2 R D' R' B2 D2 B L2 B U B U L2	R B2 L2 D' R' U' L' D' L2 D' R' U2 D2 L2 F' U2 B' L2	TRUE	1.875	2	18	26	15.34
90	B2 R B' R U2 B' U' B D U F' D R2 L B L D F2 B2 U2 R2 U L F D'	F2 D L' F' R F2 R' D2 B' D R2 D2 B' R2 B D2 F2 U2 F'	TRUE	2.046	4	19	28	15.36
91	F' B' L2 R2 B2 R D2 F' D' U2 L2 D' B' D2 B2 D2 L2 B2 R F' D2 R2 F2 R D2	R2 F U' L' U R' L2 F L2 B2 R U B2 D2 F2 U B2 D B2 D2	TRUE	2.139	3	20	30	15.35
92	L R' B' L B2 L2 F L D' U R2 F L' F2 B' D2 F2 U2 B D' F D' R2 L' U2	U R' F2 R' D2 B2 L U' B R U F' L2 U2 F R2 B' L2 F2 R2 F'	TRUE	2.405	8	21	30	15.35
93	D F' B' L F U L D' B' F' D' U' B2 F2 U2 R2 B2 D2 F2 D2 F2 U R2 U2 B2	D' U R2 F2 D2 B2 U' B' F D L' U' F' L' B F D'	TRUE	1.5	1	17	21	15.35
94	D B L B2 F' L D2 F' U2 L B U' L U B D2 R' D2 U2 L2 B U' D2 F2 D	F2 U' D2 F' U' R2 U' R2 L F D2 L' F2 B2 L' F2 B2 R D2 R	TRUE	2.561	17	20	30	15.35
95	B' F' D2 B2 F2 R2 D F2 R' L' U2 F L' B' R' L2 D' R U' F R' U' F' L B'	R L F' D2 R' U F2 D B' U' B2 D B2 U' L2 F2 L2 U2 F2	TRUE	2.061	1	19	28	15.35
96	F U R2 F2 U' B2 F' L F R2 U' D B2 R2 F2 D U2 L D F B' L2 R2 D2 R	R U' F R B L U F2 L D' F' D' L2 D L2 B2 D F2 D2 B2 D'	TRUE	1.985	1	21	28	15.35
97	L B' D' L2 D F2 D F2 D2 L2 B2 R D R D2 R' D B2 D R D2 U' L D R2	R L U L2 B2 U' B' L2 U2 L U B2 R2 U B2 U2 F2 L2 U	TRUE	2.14	1	19	29	15.34
98	B2 D2 B2 F2 L' U2 R U F' D2 F U R F2 L2 U' D L U B' F' L D R D	R' B2 U2 F' U' R' D2 F2 B D' L2 D' B2 D F2 D B2 L2 U2	TRUE	2.031	1	19	29	15.35
99	F B' R2 D' L' U F2 B2 U2 D' R' B L D R U F U' B R D U2 R2 B R2	B2 D' L B R' B' D' F2 R2 F L' D B2 L2 U' D2 B2 R2 U2 R2	TRUE	2.203	1	20	30	15.35
			100.00%	2.117	3.9	19.7	28.7	15.35